NexFlow — Full-Stack Build Pack for Replit

> Universal AI-powered B2B revenue engine. One platform replacing CRM + enrichment + voice + messaging + call intelligence + compliance + workflow automation. This document is a **complete, no-placeholder build pack** — paste it into Replit AI and build end-to-end.

- **Version:** 1.1 · April 2026 (includes NexFlow Full Blend brand identity)
- **Stack:** Next.js 15 · Node/Hono · PostgreSQL 16 · Redis 7 · Drizzle ORM · Anthropic Claude · OpenAI Whisper/TTS
- **Architecture:** Monorepo (Turborepo) — `/apps/web` (Next.js) + `/apps/api` (Hono) + `/packages/db` (Drizzle) + `/packages/ai` (agent orchestration) + `/packages/ui` (shadcn)
- **Brand:** NexFlow Full Blend — Chameleon palette · Living Mesh · Pulse rule · Flow rule (Section 14b)
- **Deployment target:** Replit (dev/staging) → Vercel + Neon + Upstash (production)

Table of Contents

---

0. How to Use This Pack in Replit

1. Create a new **Node.js** Repl (not a template — start blank).
2. Open the Replit AI sidebar (the chat panel).
3. Copy the entire block in **Section 22 (Replit AI Master Prompt)** and paste it as your first message.
4. Replit AI will scaffold the monorepo. When it asks, answer with "Use the files from the pack I'm about to paste."
5. Paste sections **3 → 20** one at a time into the chat, starting with the layout, then env, then config, then schema, then routes, then agents, then frontend.
6. After everything is in place, run the commands in **Section 21**.
7. Verify using **Section 23**.

If Replit AI drifts, the rule is: **the pack is the source of truth**. Reference section numbers in every follow-up ("Apply Section 6 exactly; no invented fields").

---

1. Product Identity

NexFlow is a **universal AI-native CRM**. It is category-agnostic and region-agnostic — any B2B sales team, any market.

- **Core promise:** One platform replacing 7 fragmented tools (CRM + enrichment + voice + messaging + call intelligence + compliance + workflow). $850/seat/year vs $12,828 legacy stack. 3.4-month payback. 68% gross margin.
- **Design principles:** Event-sourced core, AI-as-a-service, pluggable intelligence (enrichment/compliance/signals are adapters), i18n-first (RTL-aware), compliance-as-code.
- **First reference customer:** Saudi wealth management (CMA/SAMA/SIMAH/TASI/NOMU as optional plugins). Not the identity.

---

2. The 4 Systems × 17 Modules

Systems (product surface)

1. **Intelligent CRM Core** — AI queries, 360° profiles, pipeline with predictive scoring, unified activity log.
2. **Enrichment & Intelligence** — 50+ sources, waterfall enrichment, signals, NL-segments, relationship graph, playbooks.
3. **Engagement & Communication** — AI voice agent (Saudi-accented AR+EN ready), live call coaching, WhatsApp flows, email/SMS, best-time calendar, scripts.
4. **Compliance & Operations** — pluggable screening (OFAC · UN · EU · Dow Jones PEP · optional SAMA/SIMAH/MOJ), append-only audit, RBAC, encryption, analytics.

17 Modules (engineering surface — maps 1:1 to routes in `/app`)

1. Home / Command Dashboard
2. Dashboard Hub (KPIs)
3. Profile 360° (contact + company)

4. Signal Intelligence
5. Harvest (lead-gen / import)
6. Categorization Engine (segments)
7. Call Monitor (live)
8. Voice Studio (agent config)
9. WhatsApp Business
10. Scripts & Playbooks
11. Pipeline (deals)
12. Command Center (admin)
13. AI Assistant (global chat)
14. Notifications
15. Predictive Forecast
16. Best-Time Calendar
17. Analytics

---

3. Monorepo Layout

```
nexflow/
├── .replit
├── replit.nix
├── package.json
├── turbo.json
├── pnpm-workspace.yaml
├── tsconfig.base.json
├── .env.example
├── .gitignore
├── apps/
│   ├── web/                    # Next.js 15 frontend
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   │   ├── sign-in/page.tsx
│   │   │   │   └── sign-up/page.tsx
│   │   │   ├── (app)/
│   │   │   │   ├── layout.tsx
```

```
│   │   │   │   ├── page.tsx                          # 01 Home
│   │   │   │   ├── dashboard/page.tsx                # 02 Dashboard Hub
│   │   │   │   ├── contacts/page.tsx                 # list
│   │   │   │   ├── contacts/[id]/page.tsx            # 03 Profile 360°
│   │   │   │   ├── companies/page.tsx
│   │   │   │   ├── companies/[id]/page.tsx
│   │   │   │   ├── signals/page.tsx                  # 04 Signal Intel
│   │   │   │   ├── harvest/page.tsx                  # 05 Harvest
│   │   │   │   ├── segments/page.tsx                 # 06 Categorization
│   │   │   │   ├── calls/page.tsx                    # 07 Call Monitor
│   │   │   │   ├── calls/[id]/page.tsx               # live call room
│   │   │   │   ├── voice-studio/page.tsx             # 08 Voice Studio
│   │   │   │   ├── whatsapp/page.tsx                 # 09 WhatsApp
│   │   │   │   ├── scripts/page.tsx                  # 10 Scripts
│   │   │   │   ├── deals/page.tsx                    # 11 Pipeline
│   │   │   │   ├── admin/page.tsx                    # 12 Command Center
│   │   │   │   ├── assistant/page.tsx                # 13 AI Assistant
│   │   │   │   ├── notifications/page.tsx            # 14 Notifications
│   │   │   │   ├── forecast/page.tsx                 # 15 Forecast
│   │   │   │   ├── calendar/page.tsx                 # 16 Best-Time Calendar
│   │   │   │   └── analytics/page.tsx                # 17 Analytics
│   │   │   ├── api/                                  # Next route handlers (prox
│   │   │   │   └── [[...path]]/route.ts
│   │   │   ├── layout.tsx
│   │   │   ├── globals.css
│   │   │   └── not-found.tsx
│   │   ├── components/
│   │   ├── lib/
│   │   ├── next.config.mjs
│   │   ├── tailwind.config.ts
│   │   ├── postcss.config.mjs
│   │   ├── tsconfig.json
│   │   └── package.json
│   └── api/                        # Hono API server
│       ├── src/
│       │   ├── index.ts                              # entry
│       │   ├── env.ts
│       │   ├── middleware/
│       │   │   ├── auth.ts
│       │   │   ├── rls.ts
│       │   │   ├── error.ts
│       │   │   └── rate-limit.ts
│       │   ├── routes/
│       │   │   ├── contacts.ts
│       │   │   ├── companies.ts
│       │   │   ├── deals.ts
│       │   │   ├── activities.ts
```

```
│       │   │   ├── calls.ts
│       │   │   ├── signals.ts
│       │   │   ├── segments.ts
│       │   │   ├── scripts.ts
│       │   │   ├── voice-agents.ts
│       │   │   ├── whatsapp.ts
│       │   │   ├── compliance.ts
│       │   │   ├── ai.ts
│       │   │   ├── notifications.ts
│       │   │   ├── analytics.ts
│       │   │   └── integrations.ts
│       │   ├── ws/                                    # WebSocket handlers
│       │   │   ├── call-room.ts
│       │   │   └── signal-stream.ts
│       │   ├── jobs/                                  # BullMQ workers
│       │   │   ├── enrich.worker.ts
│       │   │   ├── screen.worker.ts
│       │   │   └── score.worker.ts
│       │   └── utils/
│       ├── tsconfig.json
│       └── package.json
└── packages/
    ├── db/                      # Drizzle schema + migrations
    │   ├── src/
    │   │   ├── schema.ts
    │   │   ├── client.ts
    │   │   ├── policies.sql
    │   │   └── seeds.ts
    │   ├── drizzle.config.ts
    │   └── package.json
    ├── ai/                      # Agent orchestration
    │   ├── src/
    │   │   ├── gateway.ts                         # model router + cost track
    │   │   ├── agents/
    │   │   │   ├── signal-scorer.ts
    │   │   │   ├── lead-scorer.ts
    │   │   │   ├── compliance-screener.ts
    │   │   │   ├── call-coach.ts
    │   │   │   ├── call-scorer.ts
    │   │   │   ├── segment-generator.ts
    │   │   │   ├── deal-forecaster.ts
    │   │   │   ├── daily-briefing.ts
    │   │   │   ├── ai-assistant.ts
    │   │   │   └── contact-enricher.ts
    │   │   └── prompts/
    │   └── package.json
    ├── ui/                      # shared shadcn components
```

```
│   └── package.json
└── shared/                    # zod schemas, types
    ├── src/
    │   ├── schemas.ts
    │   └── types.ts
    └── package.json
```

---

4. Environment Variables (complete)

Create `.env.example` at repo root. Replit reads from Secrets — mirror these keys in the Secrets tab.

```
# ── Core ──
NODE_ENV=development
APP_URL=http://localhost:3000
API_URL=http://localhost:4000
WS_URL=ws://localhost:4000

# ── Database (Neon / Replit PG / local) ──
DATABASE_URL=postgresql://user:password@host:5432/nexflow?sslmode=require

# ── Redis (Upstash or Replit) ──
REDIS_URL=rediss://default:password@host:6379

# ── Auth (Clerk) ──
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_xxx
CLERK_SECRET_KEY=sk_test_xxx
CLERK_WEBHOOK_SECRET=whsec_xxx

# ── AI ──
ANTHROPIC_API_KEY=sk-ant-xxx
OPENAI_API_KEY=sk-xxx
AI_DEFAULT_MODEL=claude-sonnet-4-6
AI_FAST_MODEL=claude-haiku-4-5-20251001
AI_DEEP_MODEL=claude-opus-4-7

# ── Voice ──
OPENAI_TTS_MODEL=tts-1-hd
OPENAI_WHISPER_MODEL=whisper-1

# ── Messaging / Telephony ──
TWILIO_ACCOUNT_SID=ACxxxx
TWILIO_AUTH_TOKEN=xxxx
TWILIO_PHONE_NUMBER=+15551234567
```

```
WHATSAPP_PHONE_NUMBER_ID=xxxx
WHATSAPP_BUSINESS_ACCOUNT_ID=xxxx
WHATSAPP_ACCESS_TOKEN=xxxx
WHATSAPP_VERIFY_TOKEN=nexflow_verify_2026

# ── Email ──
SENDGRID_API_KEY=SG.xxxx
EMAIL_FROM=no-reply@nexflow.app

# ── Enrichment (pluggable) ──
APOLLO_API_KEY=
CLEARBIT_API_KEY=
HUNTER_API_KEY=
ZOOMINFO_CLIENT_ID=
ZOOMINFO_CLIENT_SECRET=
PEOPLEDATALABS_API_KEY=

# ── Compliance screening (pluggable) ──
DOW_JONES_API_KEY=
REFINITIV_WORLDCHECK_API_KEY=
COMPLYADVANTAGE_API_KEY=

# ── Observability ──
SENTRY_DSN=
OTEL_EXPORTER_OTLP_ENDPOINT=

# ── Security ──
JWT_SECRET=change_me_32+_chars_minimum_xxxxxxxxxxxxxxxx
ENCRYPTION_KEY=change_me_exactly_32_chars_aes256
```

---

5. Root Configuration Files (complete)

`package.json` (root)

```
{
  "name": "nexflow",
  "private": true,
  "version": "1.0.0",
  "packageManager": "pnpm@9.12.0",
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "start": "turbo run start",
```

```
    "lint": "turbo run lint",
    "typecheck": "turbo run typecheck",
    "db:generate": "pnpm --filter @nexflow/db drizzle-kit generate",
    "db:migrate": "pnpm --filter @nexflow/db drizzle-kit migrate",
    "db:push": "pnpm --filter @nexflow/db drizzle-kit push",
    "db:studio": "pnpm --filter @nexflow/db drizzle-kit studio",
    "db:seed": "pnpm --filter @nexflow/db tsx src/seeds.ts"
  },
  "devDependencies": {
    "turbo": "^2.3.0",
    "typescript": "^5.6.3"
  },
  "engines": {
    "node": ">=20.0.0",
    "pnpm": ">=9.0.0"
  }
}
```

pnpm-workspace.yaml

```
packages:
  - "apps/*"
  - "packages/*"
```

turbo.json

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "!.next/cache/**", "dist/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "start": {
      "cache": false,
      "persistent": true
    },
    "lint": {},
    "typecheck": {}
  }
```

```
}
```

tsconfig.base.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023", "DOM"],
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "baseUrl": ".",
    "paths": {
      "@nexflow/db": ["packages/db/src"],
      "@nexflow/db/*": ["packages/db/src/*"],
      "@nexflow/ai": ["packages/ai/src"],
      "@nexflow/ai/*": ["packages/ai/src/*"],
      "@nexflow/ui": ["packages/ui/src"],
      "@nexflow/shared": ["packages/shared/src"],
      "@nexflow/shared/*": ["packages/shared/src/*"]
    }
  }
}
```

.gitignore

```
node_modules/
.next/
dist/
.turbo/
.env
.env.local
*.log
.DS_Store
.replit.local
```

---

6. Database — Drizzle Schema (complete, all 16 tables)

packages/db/package.json

```
{
  "name": "@nexflow/db",
  "version": "1.0.0",
  "type": "module",
  "main": "./src/index.ts",
  "types": "./src/index.ts",
  "scripts": {
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "drizzle-orm": "^0.36.0",
    "postgres": "^3.4.5"
  },
  "devDependencies": {
    "drizzle-kit": "^0.28.0",
    "tsx": "^4.19.0",
    "typescript": "^5.6.3"
  }
}
```

packages/db/drizzle.config.ts

```
import { defineConfig } from "drizzle-kit";

export default defineConfig({
  schema: "./src/schema.ts",
  out: "./drizzle",
  dialect: "postgresql",
  dbCredentials: { url: process.env.DATABASE_URL! },
  strict: true,
  verbose: true,
});
```

packages/db/src/client.ts

```
import { drizzle } from "drizzle-orm/postgres-js";
import postgres from "postgres";
import * as schema from "./schema";
```

```
const connectionString = process.env.DATABASE_URL!;
const queryClient = postgres(connectionString, { max: 20 });
export const db = drizzle(queryClient, { schema });

// Set tenant context per request
export async function withOrg<T>(orgId: string, fn: () => Promise<T>): Promise<T>
  return await db.transaction(async (tx) => {
    await tx.execute(
      `SELECT set_config('app.current_org_id', '${orgId}', true)`
    );
    return await fn();
  });
}
```

`packages/db/src/schema.ts` — ALL 16 TABLES

```
import {
  pgTable, uuid, text, timestamp, integer, bigint, boolean, jsonb,
  pgEnum, primaryKey, index, uniqueIndex, check, foreignKey,
} from "drizzle-orm/pg-core";
import { relations, sql } from "drizzle-orm";

// ─────────────────────────────────────────────────────────────
// 01 — organizations
// ─────────────────────────────────────────────────────────────
export const organizations = pgTable("organizations", {
  id: uuid("id").primaryKey().defaultRandom(),
  name: text("name").notNull(),
  slug: text("slug").notNull().unique(),
  region: text("region").notNull().default("global"),       // global | ksa | uae
  plan: text("plan").notNull().default("trial"),            // trial | pro | ente
  settings: jsonb("settings").$type<{
    currency: string;
    locale: string;
    complianceLists: string[];
    enrichmentProviders: string[];
  }>().notNull().default(sql`'{}'::jsonb`),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
});

// ─────────────────────────────────────────────────────────────
// 02 — users
// ─────────────────────────────────────────────────────────────
export const users = pgTable("users", {
  id: uuid("id").primaryKey().defaultRandom(),
```

```
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  authId: text("auth_id").notNull().unique(),  // Clerk user id
  email: text("email").notNull(),
  fullName: text("full_name").notNull(),
  role: text("role").notNull().default("member"),  // admin | manager | member |
  locale: text("locale").notNull().default("en"),
  phone: text("phone"),
  avatarUrl: text("avatar_url"),
  isActive: boolean("is_active").notNull().default(true),
  lastSeenAt: timestamp("last_seen_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("users_org_idx").on(t.orgId),
  emailIdx: uniqueIndex("users_email_idx").on(t.email),
}));

// ─────────────────────────────────────────────────────────────
// 03 — companies
// ─────────────────────────────────────────────────────────────
export const companies = pgTable("companies", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  name: text("name").notNull(),
  nameI18n: jsonb("name_i18n").$type<Record<string, string>>(),
  domain: text("domain"),
  industry: text("industry"),
  sector: text("sector"),
  country: text("country"),
  region: text("region"),
  city: text("city"),
  employeeCount: integer("employee_count"),
  revenueAnnual: bigint("revenue_annual", { mode: "number" }),  // smallest curre
  revenueCurrency: text("revenue_currency").default("USD"),
  foundedYear: integer("founded_year"),
  registrationNumber: text("registration_number"),   // CR / VAT / EIN
  linkedinUrl: text("linkedin_url"),
  websiteUrl: text("website_url"),
  logoUrl: text("logo_url"),
  description: text("description"),
  tags: text("tags").array(),
  enrichmentSignals: jsonb("enrichment_signals").$type<Record<string, unknown>>()
  enrichmentLastRunAt: timestamp("enrichment_last_run_at", { withTimezone: true }
  ownerUserId: uuid("owner_user_id").references(() => users.id),
  customFields: jsonb("custom_fields").$type<Record<string, unknown>>(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
```

```
}, (t) => ({
  orgIdx: index("companies_org_idx").on(t.orgId),
  domainIdx: index("companies_domain_idx").on(t.domain),
  nameIdx: index("companies_name_idx").on(t.name),
}));

// ─────────────────────────────────────────────────────────────
// 04 — contacts (PRIMARY CRM ENTITY)
// ─────────────────────────────────────────────────────────────
export const contacts = pgTable("contacts", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  companyId: uuid("company_id").references(() => companies.id, { onDelete: "set n
  fullName: text("full_name").notNull(),
  displayNameI18n: jsonb("display_name_i18n").$type<Record<string, string>>(),
  firstName: text("first_name"),
  lastName: text("last_name"),
  title: text("title"),
  titleI18n: jsonb("title_i18n").$type<Record<string, string>>(),
  email: text("email"),
  phone: text("phone"),
  whatsapp: text("whatsapp"),
  linkedinUrl: text("linkedin_url"),
  country: text("country"),
  city: text("city"),
  languages: text("languages").array(),
  leadScore: integer("lead_score").default(0),
  lifecycleStage: text("lifecycle_stage").notNull().default("lead"),
    // lead | contacted | qualified | meeting | proposal | customer | disqualifie
  complianceStatus: text("compliance_status").notNull().default("pending"),
    // pending | clear | flagged | blocked | not_required
  complianceLastScreenedAt: timestamp("compliance_last_screened_at", { withTimezo
  enrichmentSignals: jsonb("enrichment_signals").$type<Record<string, unknown>>()
  enrichmentLastRunAt: timestamp("enrichment_last_run_at", { withTimezone: true }
  enrichmentConfidence: text("enrichment_confidence"),   // low | medium | high
  doNotContact: boolean("do_not_contact").notNull().default(false),
  ownerUserId: uuid("owner_user_id").references(() => users.id),
  source: text("source"),    // manual | import | harvest | signal | webhook
  customFields: jsonb("custom_fields").$type<Record<string, unknown>>(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("contacts_org_idx").on(t.orgId),
  companyIdx: index("contacts_company_idx").on(t.companyId),
  emailIdx: index("contacts_email_idx").on(t.email),
  scoreIdx: index("contacts_score_idx").on(t.leadScore),
  lifecycleIdx: index("contacts_lifecycle_idx").on(t.lifecycleStage),
```

```
  scoreCheck: check("contacts_score_range", sql`${t.leadScore} BETWEEN 0 AND 100`
}));

// ─────────────────────────────────────────────────────────────
// 05 — signals
// ─────────────────────────────────────────────────────────────
export const signals = pgTable("signals", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  type: text("type").notNull(),
    // funding | exec_change | ipo | acquisition | layoff | expansion | partnersh
  companyId: uuid("company_id").references(() => companies.id, { onDelete: "casca
  contactId: uuid("contact_id").references(() => contacts.id, { onDelete: "cascad
  headline: text("headline").notNull(),
  body: text("body"),
  sourceUrl: text("source_url"),
  sourceName: text("source_name"),
  severity: integer("severity").default(50),  // 0-100
  aiScore: integer("ai_score"),
  aiSummary: text("ai_summary"),
  publishedAt: timestamp("published_at", { withTimezone: true }),
  seenBy: uuid("seen_by").array(),
  dismissedBy: uuid("dismissed_by").array(),
  metadata: jsonb("metadata").$type<Record<string, unknown>>(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("signals_org_idx").on(t.orgId),
  companyIdx: index("signals_company_idx").on(t.companyId),
  typeIdx: index("signals_type_idx").on(t.type),
  publishedIdx: index("signals_published_idx").on(t.publishedAt),
}));

// ─────────────────────────────────────────────────────────────
// 06 — deals
// ─────────────────────────────────────────────────────────────
export const deals = pgTable("deals", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  companyId: uuid("company_id").references(() => companies.id, { onDelete: "set n
  primaryContactId: uuid("primary_contact_id").references(() => contacts.id, { on
  name: text("name").notNull(),
  stage: text("stage").notNull().default("discovery"),
    // discovery | qualification | proposal | negotiation | closed_won | closed_l
  amount: bigint("amount", { mode: "number" }).default(0),  // smallest currency
  currency: text("currency").default("USD"),
  probability: integer("probability").default(20),  // 0-100
  expectedCloseDate: timestamp("expected_close_date", { withTimezone: true }),
```

```
  actualCloseDate: timestamp("actual_close_date", { withTimezone: true }),
  lostReason: text("lost_reason"),
  aiForecast: jsonb("ai_forecast").$type<{
    probability: number;
    reasoning: string;
    risks: string[];
    nextActions: string[];
  }>(),
  ownerUserId: uuid("owner_user_id").references(() => users.id),
  customFields: jsonb("custom_fields").$type<Record<string, unknown>>(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("deals_org_idx").on(t.orgId),
  stageIdx: index("deals_stage_idx").on(t.stage),
  ownerIdx: index("deals_owner_idx").on(t.ownerUserId),
  probCheck: check("deals_prob_range", sql`${t.probability} BETWEEN 0 AND 100`),
}));

// ─────────────────────────────────────────────────────────────
// 07 — activities (unified log)
// ─────────────────────────────────────────────────────────────
export const activities = pgTable("activities", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  type: text("type").notNull(),
    // call | email | whatsapp | sms | meeting | note | task | signal_reaction
  contactId: uuid("contact_id").references(() => contacts.id, { onDelete: "cascad
  companyId: uuid("company_id").references(() => companies.id, { onDelete: "casca
  dealId: uuid("deal_id").references(() => deals.id, { onDelete: "cascade" }),
  userId: uuid("user_id").references(() => users.id),
  direction: text("direction"),  // inbound | outbound | internal
  subject: text("subject"),
  body: text("body"),
  durationSeconds: integer("duration_seconds"),
  occurredAt: timestamp("occurred_at", { withTimezone: true }).notNull().defaultN
  metadata: jsonb("metadata").$type<Record<string, unknown>>(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("activities_org_idx").on(t.orgId),
  contactIdx: index("activities_contact_idx").on(t.contactId),
  typeIdx: index("activities_type_idx").on(t.type),
  occurredIdx: index("activities_occurred_idx").on(t.occurredAt),
}));

// ─────────────────────────────────────────────────────────────
// 08 — calls (extends activities)
```

```
// ─────────────────────────────────────────────────────────────
export const calls = pgTable("calls", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  activityId: uuid("activity_id").references(() => activities.id, { onDelete: "ca
  contactId: uuid("contact_id").references(() => contacts.id, { onDelete: "cascad
  userId: uuid("user_id").references(() => users.id),
  voiceAgentId: uuid("voice_agent_id"),
  twilioCallSid: text("twilio_call_sid"),
  status: text("status").notNull().default("queued"),
    // queued | ringing | in_progress | completed | failed | no_answer | busy
  direction: text("direction").notNull().default("outbound"),
  fromNumber: text("from_number"),
  toNumber: text("to_number"),
  durationSeconds: integer("duration_seconds"),
  recordingUrl: text("recording_url"),
  transcript: jsonb("transcript").$type<Array<{
    speaker: "agent" | "contact" | "ai";
    text: string;
    ts: number;
    sentiment?: number;
  }>>(),
  aiScore: integer("ai_score"),
  aiSummary: text("ai_summary"),
  aiCoachingNotes: jsonb("ai_coaching_notes").$type<string[]>(),
  nextActions: jsonb("next_actions").$type<string[]>(),
  startedAt: timestamp("started_at", { withTimezone: true }),
  endedAt: timestamp("ended_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("calls_org_idx").on(t.orgId),
  statusIdx: index("calls_status_idx").on(t.status),
  sidIdx: uniqueIndex("calls_sid_idx").on(t.twilioCallSid),
}));

// ─────────────────────────────────────────────────────────────
// 09 — compliance_screenings (APPEND-ONLY)
// ─────────────────────────────────────────────────────────────
export const complianceScreenings = pgTable("compliance_screenings", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  contactId: uuid("contact_id").references(() => contacts.id, { onDelete: "cascad
  companyId: uuid("company_id").references(() => companies.id, { onDelete: "casca
  triggeredBy: uuid("triggered_by").references(() => users.id),
  trigger: text("trigger").notNull(),   // manual | onboarding | periodic | signa
  lists: text("lists").array().notNull(),  // ofac_sdn | un_consolidated | eu_san
  result: text("result").notNull(),   // clear | flagged | blocked
```

```
  matches: jsonb("matches").$type<Array<{
    list: string;
    score: number;
    matchedName: string;
    matchedFields: string[];
    sourceUrl?: string;
  }>>().notNull().default(sql`'[]'::jsonb`),
  rawResponse: jsonb("raw_response").$type<Record<string, unknown>>(),
  notes: text("notes"),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("screenings_org_idx").on(t.orgId),
  contactIdx: index("screenings_contact_idx").on(t.contactId),
  createdIdx: index("screenings_created_idx").on(t.createdAt),
}));

// ─────────────────────────────────────────────────────────────
// 10 — segments
// ─────────────────────────────────────────────────────────────
export const segments = pgTable("segments", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  name: text("name").notNull(),
  description: text("description"),
  naturalLanguageQuery: text("natural_language_query"),
  filterJson: jsonb("filter_json").$type<Record<string, unknown>>().notNull(),
  entityType: text("entity_type").notNull().default("contact"),  // contact | com
  memberCount: integer("member_count").default(0),
  createdBy: uuid("created_by").references(() => users.id),
  isShared: boolean("is_shared").default(false),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("segments_org_idx").on(t.orgId),
}));

// ─────────────────────────────────────────────────────────────
// 11 — scripts
// ─────────────────────────────────────────────────────────────
export const scripts = pgTable("scripts", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  name: text("name").notNull(),
  category: text("category").notNull(),  // cold_call | discovery | objection | c
  locale: text("locale").notNull().default("en"),
  content: text("content").notNull(),
  variables: jsonb("variables").$type<string[]>(),
```

```
  objectionHandlers: jsonb("objection_handlers").$type<Array<{
    objection: string;
    response: string;
  }>>(),
  usageCount: integer("usage_count").default(0),
  avgScore: integer("avg_score"),
  createdBy: uuid("created_by").references(() => users.id),
  isActive: boolean("is_active").default(true),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("scripts_org_idx").on(t.orgId),
}));

// ─────────────────────────────────────────────────────────────
// 12 — voice_agents
// ─────────────────────────────────────────────────────────────
export const voiceAgents = pgTable("voice_agents", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  name: text("name").notNull(),
  persona: text("persona"),
  voice: text("voice").notNull().default("nova"),  // nova | onyx | shimmer | ech
  language: text("language").notNull().default("en"),
  dialect: text("dialect"),  // en-US | en-GB | ar-SA | ar-EG | ar-AE
  systemPrompt: text("system_prompt").notNull(),
  initialGreeting: text("initial_greeting"),
  objectives: jsonb("objectives").$type<string[]>(),
  knowledgeBaseIds: uuid("knowledge_base_ids").array(),
  temperature: integer("temperature").default(70),  // stored ×100
  maxTurns: integer("max_turns").default(20),
  isActive: boolean("is_active").default(true),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("voice_agents_org_idx").on(t.orgId),
}));

// ─────────────────────────────────────────────────────────────
// 13 — messaging_flows
// ─────────────────────────────────────────────────────────────
export const messagingFlows = pgTable("messaging_flows", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  name: text("name").notNull(),
  channel: text("channel").notNull(),  // whatsapp | sms | email
  triggerType: text("trigger_type").notNull(),  // manual | signal | stage_change
```

```
  triggerConfig: jsonb("trigger_config").$type<Record<string, unknown>>(),
  nodes: jsonb("nodes").$type<Array<{
    id: string;
    type: "message" | "delay" | "branch" | "action" | "end";
    config: Record<string, unknown>;
    next?: string | string[];
  }>>().notNull(),
  edges: jsonb("edges").$type<Array<{ from: string; to: string; condition?: strin
  isActive: boolean("is_active").default(false),
  stats: jsonb("stats").$type<{
    entered: number;
    completed: number;
    replied: number;
  }>(),
  createdBy: uuid("created_by").references(() => users.id),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("flows_org_idx").on(t.orgId),
}));

// ─────────────────────────────────────────────────────────────
// 14 — ai_conversations
// ─────────────────────────────────────────────────────────────
export const aiConversations = pgTable("ai_conversations", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "casca
  title: text("title"),
  contextType: text("context_type"),    // contact | company | deal | global
  contextId: uuid("context_id"),
  messages: jsonb("messages").$type<Array<{
    role: "user" | "assistant" | "system" | "tool";
    content: string;
    toolCalls?: unknown[];
    ts: number;
  }>>().notNull().default(sql`'[]'::jsonb`),
  model: text("model"),
  totalTokensIn: integer("total_tokens_in").default(0),
  totalTokensOut: integer("total_tokens_out").default(0),
  costCents: integer("cost_cents").default(0),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgIdx: index("ai_conv_org_idx").on(t.orgId),
  userIdx: index("ai_conv_user_idx").on(t.userId),
}));
```

```
// ─────────────────────────────────────────────────────────────
// 15 — notifications
// ─────────────────────────────────────────────────────────────
export const notifications = pgTable("notifications", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "casca
  type: text("type").notNull(),  // signal | deal_stage | compliance | call_compl
  title: text("title").notNull(),
  body: text("body"),
  link: text("link"),
  entityType: text("entity_type"),
  entityId: uuid("entity_id"),
  isRead: boolean("is_read").notNull().default(false),
  readAt: timestamp("read_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  userIdx: index("notifs_user_idx").on(t.userId),
  unreadIdx: index("notifs_unread_idx").on(t.userId, t.isRead),
}));

// ─────────────────────────────────────────────────────────────
// 16 — integrations
// ─────────────────────────────────────────────────────────────
export const integrations = pgTable("integrations", {
  id: uuid("id").primaryKey().defaultRandom(),
  orgId: uuid("org_id").notNull().references(() => organizations.id, { onDelete:
  provider: text("provider").notNull(),
    // hubspot | salesforce | gmail | outlook | slack | teams | twilio | whatsapp
  name: text("name").notNull(),
  credentials: text("credentials").notNull(),  // AES-256-GCM encrypted JSON
  config: jsonb("config").$type<Record<string, unknown>>(),
  status: text("status").notNull().default("active"),  // active | paused | error
  lastSyncAt: timestamp("last_sync_at", { withTimezone: true }),
  lastErrorAt: timestamp("last_error_at", { withTimezone: true }),
  lastErrorMessage: text("last_error_message"),
  createdBy: uuid("created_by").references(() => users.id),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow
}, (t) => ({
  orgProviderIdx: uniqueIndex("integrations_org_provider_idx").on(t.orgId, t.prov
}));

// ─────────────────────────────────────────────────────────────
// Relations
// ─────────────────────────────────────────────────────────────
```

```
export const orgRelations = relations(organizations, ({ many }) => ({
  users: many(users),
  companies: many(companies),
  contacts: many(contacts),
  deals: many(deals),
}));

export const contactRelations = relations(contacts, ({ one, many }) => ({
  org: one(organizations, { fields: [contacts.orgId], references: [organizations.
  company: one(companies, { fields: [contacts.companyId], references: [companies.
  owner: one(users, { fields: [contacts.ownerUserId], references: [users.id] }),
  activities: many(activities),
  screenings: many(complianceScreenings),
}));

export const companyRelations = relations(companies, ({ one, many }) => ({
  org: one(organizations, { fields: [companies.orgId], references: [organizations
  owner: one(users, { fields: [companies.ownerUserId], references: [users.id] }),
  contacts: many(contacts),
  deals: many(deals),
}));

export const dealRelations = relations(deals, ({ one }) => ({
  org: one(organizations, { fields: [deals.orgId], references: [organizations.id]
  company: one(companies, { fields: [deals.companyId], references: [companies.id]
  primaryContact: one(contacts, { fields: [deals.primaryContactId], references: [
  owner: one(users, { fields: [deals.ownerUserId], references: [users.id] }),
}));
```

packages/db/src/index.ts

```
export * from "./schema";
export * from "./client";
```

---

7. Database — Policies, Triggers, Indexes, Seeds

`packages/db/src/policies.sql` — run once after `db:push`

```
-- Enable Row-Level Security on every tenant-scoped table
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE companies ENABLE ROW LEVEL SECURITY;
ALTER TABLE contacts ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE signals ENABLE ROW LEVEL SECURITY;
ALTER TABLE deals ENABLE ROW LEVEL SECURITY;
ALTER TABLE activities ENABLE ROW LEVEL SECURITY;
ALTER TABLE calls ENABLE ROW LEVEL SECURITY;
ALTER TABLE compliance_screenings ENABLE ROW LEVEL SECURITY;
ALTER TABLE segments ENABLE ROW LEVEL SECURITY;
ALTER TABLE scripts ENABLE ROW LEVEL SECURITY;
ALTER TABLE voice_agents ENABLE ROW LEVEL SECURITY;
ALTER TABLE messaging_flows ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE notifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE integrations ENABLE ROW LEVEL SECURITY;

-- Template policy per table (repeat for each)
CREATE POLICY org_isolation_contacts ON contacts
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_companies ON companies
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_deals ON deals
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_activities ON activities
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_calls ON calls
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_signals ON signals
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_screenings ON compliance_screenings
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_segments ON segments
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_scripts ON scripts
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_voice_agents ON voice_agents
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_flows ON messaging_flows
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_ai_conv ON ai_conversations
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_notifs ON notifications
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_integrations ON integrations
  USING (org_id = current_setting('app.current_org_id', true)::uuid);
CREATE POLICY org_isolation_users ON users
  USING (org_id = current_setting('app.current_org_id', true)::uuid);

-- Append-only enforcement on compliance_screenings
CREATE OR REPLACE FUNCTION block_compliance_modification()
```

```
RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'compliance_screenings is append-only. UPDATE/DELETE blocked.';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER no_update_compliance BEFORE UPDATE ON compliance_screenings
  FOR EACH ROW EXECUTE FUNCTION block_compliance_modification();
CREATE TRIGGER no_delete_compliance BEFORE DELETE ON compliance_screenings
  FOR EACH ROW EXECUTE FUNCTION block_compliance_modification();

-- updated_at auto-touch
CREATE OR REPLACE FUNCTION touch_updated_at()
RETURNS trigger AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DO $$ DECLARE t text;
BEGIN
  FOR t IN SELECT unnest(ARRAY['organizations','users','companies','contacts','de
  LOOP
    EXECUTE format('DROP TRIGGER IF EXISTS touch_%s ON %I', t, t);
    EXECUTE format('CREATE TRIGGER touch_%s BEFORE UPDATE ON %I FOR EACH ROW EXEC
  END LOOP;
END $$;
```

packages/db/src/seeds.ts

```
import { db } from "./client";
import { organizations, users, scripts, voiceAgents } from "./schema";

async function main() {
  const [org] = await db.insert(organizations).values({
    name: "NexFlow Demo Org",
    slug: "nexflow-demo",
    region: "global",
    plan: "pro",
    settings: {
      currency: "USD",
      locale: "en",
      complianceLists: ["ofac_sdn", "un_consolidated", "eu_sanctions", "dj_pep"],
      enrichmentProviders: ["apollo", "clearbit", "hunter"],
    },
```

```
  }).returning();

  const [admin] = await db.insert(users).values({
    orgId: org.id,
    authId: "seed_admin",
    email: "admin@nexflow.app",
    fullName: "Demo Admin",
    role: "admin",
    locale: "en",
  }).returning();

  await db.insert(scripts).values([
    {
      orgId: org.id,
      name: "Cold Call – Discovery",
      category: "cold_call",
      locale: "en",
      content: `Hi {contact.firstName}, this is {user.fullName} from {org.name}.
The reason for my call — I noticed {signal.headline}. Teams like yours typically
Would a 15-minute call next week make sense to explore if we can help?`,
      variables: ["contact.firstName", "user.fullName", "org.name", "signal.headl
      objectionHandlers: [
        { objection: "Not interested", response: "Totally fair. Just so I stop —
        { objection: "Send me information", response: "Happy to. Quick question s
      ],
      createdBy: admin.id,
      isActive: true,
    },
  ]);

  await db.insert(voiceAgents).values({
    orgId: org.id,
    name: "Default Outbound Agent",
    persona: "Friendly, consultative B2B SDR",
    voice: "nova",
    language: "en",
    dialect: "en-US",
    systemPrompt: `You are an outbound sales development rep for {{org.name}}.
Your goal: book a 15-minute discovery meeting.
Be warm, curious, never pushy. Ask one question at a time.
If asked what we do, describe it in one sentence using {{org.pitch}}.
If the contact asks for information, confirm email and say it will be sent.
Never promise pricing. Never commit to delivery dates.`,
    initialGreeting: "Hi, is this {{contact.firstName}}? I caught you at an okay
    objectives: ["Qualify interest", "Book discovery meeting", "Confirm email"],
    isActive: true,
  });
```

```
  console.log("✓ Seeded org, admin, script, voice agent");
}

main().then(() => process.exit(0)).catch((e) => {
  console.error(e);
  process.exit(1);
});
```

---

8. Backend API — Hono Server (complete, all routes)

apps/api/package.json

```
{
  "name": "@nexflow/api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.30.0",
    "@hono/node-server": "^1.13.0",
    "@hono/zod-validator": "^0.4.0",
    "@nexflow/ai": "workspace:*",
    "@nexflow/db": "workspace:*",
    "@nexflow/shared": "workspace:*",
    "bullmq": "^5.25.0",
    "drizzle-orm": "^0.36.0",
    "hono": "^4.6.0",
    "ioredis": "^5.4.1",
    "jsonwebtoken": "^9.0.2",
    "openai": "^4.70.0",
    "postgres": "^3.4.5",
    "twilio": "^5.3.0",
    "ws": "^8.18.0",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/jsonwebtoken": "^9.0.7",
```

```
    "@types/node": "^22.9.0",
    "@types/ws": "^8.5.13",
    "tsx": "^4.19.0",
    "typescript": "^5.6.3"
  }
}
```

`apps/api/src/index.ts` — **entry**

```
import { serve } from "@hono/node-server";
import { Hono } from "hono";
import { cors } from "hono/cors";
import { logger } from "hono/logger";
import { secureHeaders } from "hono/secure-headers";
import { errorHandler } from "./middleware/error";
import { authMiddleware } from "./middleware/auth";
import { rlsMiddleware } from "./middleware/rls";
import contactsRouter from "./routes/contacts";
import companiesRouter from "./routes/companies";
import dealsRouter from "./routes/deals";
import activitiesRouter from "./routes/activities";
import callsRouter from "./routes/calls";
import signalsRouter from "./routes/signals";
import segmentsRouter from "./routes/segments";
import scriptsRouter from "./routes/scripts";
import voiceAgentsRouter from "./routes/voice-agents";
import whatsappRouter from "./routes/whatsapp";
import complianceRouter from "./routes/compliance";
import aiRouter from "./routes/ai";
import notificationsRouter from "./routes/notifications";
import analyticsRouter from "./routes/analytics";
import integrationsRouter from "./routes/integrations";

const app = new Hono();

app.use("*", logger());
app.use("*", secureHeaders());
app.use("*", cors({
  origin: [process.env.APP_URL!],
  credentials: true,
}));

app.get("/health", (c) => c.json({ status: "ok", time: new Date().toISOString() }

// public webhooks (no auth)
app.route("/webhooks/whatsapp", whatsappRouter);
```

```
// protected
app.use("/v1/*", authMiddleware);
app.use("/v1/*", rlsMiddleware);

app.route("/v1/contacts", contactsRouter);
app.route("/v1/companies", companiesRouter);
app.route("/v1/deals", dealsRouter);
app.route("/v1/activities", activitiesRouter);
app.route("/v1/calls", callsRouter);
app.route("/v1/signals", signalsRouter);
app.route("/v1/segments", segmentsRouter);
app.route("/v1/scripts", scriptsRouter);
app.route("/v1/voice-agents", voiceAgentsRouter);
app.route("/v1/compliance", complianceRouter);
app.route("/v1/ai", aiRouter);
app.route("/v1/notifications", notificationsRouter);
app.route("/v1/analytics", analyticsRouter);
app.route("/v1/integrations", integrationsRouter);

app.onError(errorHandler);

const port = Number(process.env.PORT || 4000);
console.log(`▲ NexFlow API listening on :${port}`);
serve({ fetch: app.fetch, port });
```

apps/api/src/middleware/auth.ts

```
import { createMiddleware } from "hono/factory";
import jwt from "jsonwebtoken";
import { db } from "@nexflow/db";
import { users } from "@nexflow/db";
import { eq } from "drizzle-orm";

export const authMiddleware = createMiddleware(async (c, next) => {
  const authHeader = c.req.header("authorization");
  if (!authHeader?.startsWith("Bearer ")) return c.json({ error: "unauthorized" }

  const token = authHeader.slice(7);
  let decoded: { sub: string; orgId?: string };
  try {
    decoded = jwt.verify(token, process.env.JWT_SECRET!) as any;
  } catch {
    return c.json({ error: "invalid_token" }, 401);
  }
```

```
  const user = await db.query.users.findFirst({ where: eq(users.authId, decoded.s
  if (!user || !user.isActive) return c.json({ error: "user_inactive" }, 401);

  c.set("user", user);
  c.set("orgId", user.orgId);
  await next();
});
```

apps/api/src/middleware/rls.ts

```
import { createMiddleware } from "hono/factory";
import { db } from "@nexflow/db";
import { sql } from "drizzle-orm";

export const rlsMiddleware = createMiddleware(async (c, next) => {
  const orgId = c.get("orgId") as string;
  await db.execute(sql`SELECT set_config('app.current_org_id', ${orgId}, false)`)
  await next();
});
```

apps/api/src/middleware/error.ts

```
import type { ErrorHandler } from "hono";
import { ZodError } from "zod";

export const errorHandler: ErrorHandler = (err, c) => {
  console.error(err);
  if (err instanceof ZodError) {
    return c.json({ error: "validation_error", issues: err.issues }, 400);
  }
  return c.json({ error: "internal_error", message: err.message }, 500);
};
```

apps/api/src/routes/contacts.ts — representative pattern (applies to all resources)

```
import { Hono } from "hono";
import { zValidator } from "@hono/zod-validator";
import { z } from "zod";
import { db, contacts } from "@nexflow/db";
import { and, eq, desc, ilike, or } from "drizzle-orm";
import { enqueueEnrich } from "../jobs/enqueue";
import { enqueueScreen } from "../jobs/enqueue";
```

```
const app = new Hono();

const ContactCreate = z.object({
  fullName: z.string().min(1),
  firstName: z.string().optional(),
  lastName: z.string().optional(),
  email: z.string().email().optional(),
  phone: z.string().optional(),
  whatsapp: z.string().optional(),
  title: z.string().optional(),
  companyId: z.string().uuid().optional(),
  country: z.string().optional(),
  languages: z.array(z.string()).optional(),
  source: z.string().optional(),
});

// GET /v1/contacts?search=&stage=&ownerId=&limit=&cursor=
app.get("/", async (c) => {
  const { search, stage, ownerId, limit = "25", cursor } = c.req.query();
  const orgId = c.get("orgId") as string;
  const l = Math.min(Number(limit), 100);

  const where = and(
    eq(contacts.orgId, orgId),
    search ? or(ilike(contacts.fullName, `%${search}%`), ilike(contacts.email, `%
    stage ? eq(contacts.lifecycleStage, stage) : undefined,
    ownerId ? eq(contacts.ownerUserId, ownerId) : undefined
  );

  const rows = await db.select().from(contacts)
    .where(where)
    .orderBy(desc(contacts.updatedAt))
    .limit(l + 1);

  const hasMore = rows.length > l;
  const items = hasMore ? rows.slice(0, l) : rows;
  return c.json({ items, nextCursor: hasMore ? items.at(-1)!.updatedAt : null });
});

app.get("/:id", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  const row = await db.query.contacts.findFirst({
    where: and(eq(contacts.id, id), eq(contacts.orgId, orgId)),
    with: { company: true, owner: true },
  });
```

```
  if (!row) return c.json({ error: "not_found" }, 404);
  return c.json(row);
});

app.post("/", zValidator("json", ContactCreate), async (c) => {
  const data = c.req.valid("json");
  const orgId = c.get("orgId") as string;
  const user = c.get("user") as { id: string };

  const [row] = await db.insert(contacts).values({
    ...data,
    orgId,
    ownerUserId: user.id,
    source: data.source ?? "manual",
  }).returning();

  await enqueueEnrich({ contactId: row.id, orgId });
  await enqueueScreen({ contactId: row.id, orgId });

  return c.json(row, 201);
});

app.patch("/:id", zValidator("json", ContactCreate.partial()), async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  const data = c.req.valid("json");
  const [row] = await db.update(contacts).set({ ...data, updatedAt: new Date() })
    .where(and(eq(contacts.id, id), eq(contacts.orgId, orgId)))
    .returning();
  if (!row) return c.json({ error: "not_found" }, 404);
  return c.json(row);
});

app.delete("/:id", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  await db.delete(contacts).where(and(eq(contacts.id, id), eq(contacts.orgId, org
  return c.json({ ok: true });
});

app.post("/:id/rescreen", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  await enqueueScreen({ contactId: id, orgId, trigger: "manual" });
  return c.json({ queued: true });
});
```

```
app.post("/:id/enrich", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  await enqueueEnrich({ contactId: id, orgId });
  return c.json({ queued: true });
});

app.post("/:id/brief", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  const { generateContactBrief } = await import("@nexflow/ai/agents/daily-briefin
  const brief = await generateContactBrief({ contactId: id, orgId });
  return c.json(brief);
});

app.post("/bulk-enrich", zValidator("json", z.object({ ids: z.array(z.string().uu
  const { ids } = c.req.valid("json");
  const orgId = c.get("orgId") as string;
  await Promise.all(ids.map((contactId) => enqueueEnrich({ contactId, orgId })));
  return c.json({ queued: ids.length });
});

export default app;
```

Remaining routes — follow the identical pattern

Apply the exact same structure to the other files. The full route/verb matrix below is the contract Replit AI must produce:

File	Routes
`companies.ts`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`, `POST /:id/enrich`, `POST /:id/signals/refresh`
`deals.ts`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `PATCH /:id/stage`, `POST /:id/forecast`, `DELETE /:id`
`activities.ts`	`GET /`, `POST /`, `GET /timeline/:contactId`, `DELETE /:id`
`calls.ts`	`POST /start` (Twilio originate), `POST /:id/end`, `GET /:id`, `GET /:id/transcript`, `POST /:id/score`, Twilio webhook `POST /twilio-status`
`signals.ts`	`GET /`, `GET /stream` (SSE), `POST /ingest` (internal), `POST /:id/react`, `POST /:id/dismiss`

segments.ts	GET /, GET /:id, POST / (NL → filter), POST /:id/preview, POST /:id/export, DELETE /:id
scripts.ts	GET /, POST /, PATCH /:id, POST /:id/render (interpolate vars), DELETE /:id
voice-agents.ts	GET /, POST /, PATCH /:id, POST /:id/test-call, DELETE /:id
compliance.ts	POST /screen, GET /history/:contactId, GET /lists, POST /adverse-media
ai.ts	POST /chat (streaming), POST /segment, POST /email-draft, POST /call-summary, GET /briefing/daily, POST /deal-forecast
notifications.ts	GET /, POST /:id/read, POST /read-all, DELETE /:id
analytics.ts	GET /kpis, GET /funnel, GET /cohort, GET /leaderboard, GET /activity-heatmap
integrations.ts	GET /, POST /, DELETE /:id, POST /:id/test, POST /hubspot/sync, POST /salesforce/sync, POST /gmail/connect, POST /vibe/enrich
whatsapp.ts (webhook, no auth)	GET /webhook (verify), POST /webhook (inbound), internal POST /send

---

9. AI Agent Layer (complete, 10 agents)

packages/ai/package.json

```
{
  "name": "@nexflow/ai",
  "version": "1.0.0",
  "type": "module",
  "main": "./src/index.ts",
  "dependencies": {
    "@anthropic-ai/sdk": "^0.30.0",
    "@nexflow/db": "workspace:*",
    "openai": "^4.70.0",
    "zod": "^3.23.8"
```

```
  }
}
```

`packages/ai/src/gateway.ts` — model router + cost tracking

```ts
import Anthropic from "@anthropic-ai/sdk";
import OpenAI from "openai";

export type ModelTier = "fast" | "default" | "deep";

const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY! });
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! });

const MODELS: Record<ModelTier, string> = {
  fast: process.env.AI_FAST_MODEL ?? "claude-haiku-4-5-20251001",
  default: process.env.AI_DEFAULT_MODEL ?? "claude-sonnet-4-6",
  deep: process.env.AI_DEEP_MODEL ?? "claude-opus-4-7",
};

// Per-million tokens in USD cents
const COST: Record<string, { in: number; out: number }> = {
  "claude-haiku-4-5-20251001": { in: 100, out: 500 },
  "claude-sonnet-4-6": { in: 300, out: 1500 },
  "claude-opus-4-7": { in: 1500, out: 7500 },
};

export interface GenerateParams {
  tier?: ModelTier;
  system?: string;
  messages: Array<{ role: "user" | "assistant"; content: string }>;
  maxTokens?: number;
  temperature?: number;
  tools?: Anthropic.Tool[];
  stream?: boolean;
  jsonMode?: boolean;
}

export async function generate(params: GenerateParams) {
  const model = MODELS[params.tier ?? "default"];
  const response = await anthropic.messages.create({
    model,
    max_tokens: params.maxTokens ?? 8000,
    temperature: (params.temperature ?? 70) / 100,
    system: params.system,
    messages: params.messages,
    tools: params.tools,
```

```
  });

  // CRITICAL: filter for text blocks only
  const text = response.content
    .filter((b) => b.type === "text")
    .map((b) => (b as Anthropic.TextBlock).text)
    .join("\n");

  const cost = COST[model];
  const costCents = cost
    ? (response.usage.input_tokens * cost.in + response.usage.output_tokens * cos
    : 0;

  let json: unknown | undefined;
  if (params.jsonMode) {
    const cleaned = text.replace(/```json|```/g, "").trim();
    try { json = JSON.parse(cleaned); } catch { /* leave undefined */ }
  }

  return {
    text,
    json,
    model,
    tokensIn: response.usage.input_tokens,
    tokensOut: response.usage.output_tokens,
    costCents: Math.ceil(costCents),
  };
}

export async function* generateStream(params: GenerateParams) {
  const model = MODELS[params.tier ?? "default"];
  const stream = anthropic.messages.stream({
    model,
    max_tokens: params.maxTokens ?? 8000,
    system: params.system,
    messages: params.messages,
  });
  for await (const event of stream) {
    if (event.type === "content_block_delta" && event.delta.type === "text_delta"
      yield event.delta.text;
    }
  }
}
```

packages/ai/src/agents/signal-scorer.ts

```
import { generate } from "../gateway";
import { db, signals, contacts, companies } from "@nexflow/db";
import { eq } from "drizzle-orm";

export async function scoreSignal(signalId: string): Promise<{ score: number; sum
  const sig = await db.query.signals.findFirst({ where: eq(signals.id, signalId)
  if (!sig) throw new Error("signal not found");

  const system = `You are a B2B sales signal scoring engine.
Rate each signal 0-100 on its value as a trigger for outreach.
Consider: recency, specificity, actionability, and whether it indicates a buying
Return ONLY JSON: {"score": number, "summary": "one sentence" }`;

  const user = `Signal type: ${sig.type}
Headline: ${sig.headline}
Body: ${sig.body ?? ""}
Source: ${sig.sourceName ?? ""}`;

  const res = await generate({
    tier: "fast",
    system,
    messages: [{ role: "user", content: user }],
    jsonMode: true,
    maxTokens: 400,
  });
  const parsed = res.json as { score: number; summary: string };

  await db.update(signals).set({
    aiScore: parsed.score,
    aiSummary: parsed.summary,
  }).where(eq(signals.id, signalId));

  return parsed;
}
```

packages/ai/src/agents/lead-scorer.ts

```
import { generate } from "../gateway";
import { db, contacts, activities, signals } from "@nexflow/db";
import { eq, desc } from "drizzle-orm";

export async function scoreContact(contactId: string): Promise<number> {
  const c = await db.query.contacts.findFirst({
    where: eq(contacts.id, contactId),
    with: { company: true },
```

```
  });
  if (!c) throw new Error("contact not found");

  const recentActivities = await db.query.activities.findMany({
    where: eq(activities.contactId, contactId),
    orderBy: desc(activities.occurredAt),
    limit: 10,
  });

  const recentSignals = await db.query.signals.findMany({
    where: eq(signals.contactId, contactId),
    orderBy: desc(signals.publishedAt),
    limit: 5,
  });

  const system = `You score B2B leads 0-100. Weights:
ICP fit (industry, size, geo) — 30
Intent signals (recent triggers) — 30
Engagement (replies, meetings, calls) — 20
Seniority of role — 20
Return ONLY JSON: {"score": number, "reasoning": "..."}`;

  const user = JSON.stringify({ contact: c, activities: recentActivities, signals

  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: user }],
    jsonMode: true,
    maxTokens: 500,
  });
  const { score } = res.json as { score: number; reasoning: string };

  await db.update(contacts).set({ leadScore: score }).where(eq(contacts.id, conta
  return score;
}
```

packages/ai/src/agents/compliance-screener.ts

```
import { db, contacts, complianceScreenings } from "@nexflow/db";
import { eq } from "drizzle-orm";
import { runScreening, type ScreeningInput } from "../../../apps/api/src/complian

// thin wrapper — the heavy logic lives in the pluggable engine (Section 10)
export async function screenContact(contactId: string, orgId: string, trigger = "
  const c = await db.query.contacts.findFirst({ where: eq(contacts.id, contactId)
```

```
  if (!c) throw new Error("contact not found");

  const input: ScreeningInput = {
    fullName: c.fullName,
    country: c.country ?? undefined,
    dateOfBirth: undefined,
  };

  const result = await runScreening(input, { orgId });

  await db.insert(complianceScreenings).values({
    orgId,
    contactId,
    trigger,
    lists: result.listsUsed,
    result: result.decision,
    matches: result.matches,
    rawResponse: result.raw,
  });

  await db.update(contacts).set({
    complianceStatus: result.decision,
    complianceLastScreenedAt: new Date(),
  }).where(eq(contacts.id, contactId));

  return result;
}
```

packages/ai/src/agents/call-coach.ts

```
import { generate, generateStream } from "../gateway";

export async function suggestCoachingPrompt(opts: {
  transcript: Array<{ speaker: string; text: string }>;
  contactContext: string;
  objective: string;
}) {
  const system = `You are a live call coach feeding whispered suggestions to a re
Keep each suggestion under 12 words. Focus on the next best move.
If you hear an objection, suggest a response. If silence, suggest a question. If
  const last = opts.transcript.slice(-5).map((t) => `${t.speaker}: ${t.text}`).jo
  const user = `Objective: ${opts.objective}
Contact: ${opts.contactContext}
Recent exchange:
${last}
```

```
Suggest the rep's next move.`;
  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: user }],
    maxTokens: 100,
    temperature: 60,
  });
  return res.text.trim();
}
```

packages/ai/src/agents/call-scorer.ts

```
import { generate } from "../gateway";
import { db, calls } from "@nexflow/db";
import { eq } from "drizzle-orm";

export async function scoreCompletedCall(callId: string) {
  const call = await db.query.calls.findFirst({ where: eq(calls.id, callId) });
  if (!call || !call.transcript) throw new Error("call or transcript missing");

  const system = `Score this sales call 0-100 using:
Discovery depth — 25
Listening / talk-time ratio — 20
Objection handling — 20
Clear next step — 20
Rapport — 15
Return JSON: {"score": 0-100, "summary": "2 sentences", "coachingNotes": ["...",

  const transcriptText = call.transcript.map((t) => `${t.speaker}: ${t.text}`).jo

  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: transcriptText }],
    jsonMode: true,
    maxTokens: 1500,
  });

  const parsed = res.json as {
    score: number; summary: string; coachingNotes: string[]; nextActions: string[
  };

  await db.update(calls).set({
    aiScore: parsed.score,
    aiSummary: parsed.summary,
```

```
    aiCoachingNotes: parsed.coachingNotes,
    nextActions: parsed.nextActions,
  }).where(eq(calls.id, callId));

  return parsed;
}
```

packages/ai/src/agents/segment-generator.ts

```
import { generate } from "../gateway";

export async function naturalLanguageToFilter(nlQuery: string) {
  const system = `Translate a natural-language B2B segment description into a str
Allowed fields:
contact: fullName, title, country, city, leadScore (0-100), lifecycleStage, compl
company: name, industry, sector, country, employeeCount, revenueAnnual, foundedYe
signals: type, severity

Return JSON:
{"entity": "contact"|"company", "filters": [{"field":"...", "op":"eq|neq|gt|gte|l

  const res = await generate({
    tier: "deep",
    system,
    messages: [{ role: "user", content: nlQuery }],
    jsonMode: true,
    maxTokens: 1500,
  });
  return res.json;
}
```

packages/ai/src/agents/deal-forecaster.ts

```
import { generate } from "../gateway";
import { db, deals, activities } from "@nexflow/db";
import { eq, desc } from "drizzle-orm";

export async function forecastDeal(dealId: string) {
  const d = await db.query.deals.findFirst({
    where: eq(deals.id, dealId),
    with: { company: true, primaryContact: true },
  });
  if (!d) throw new Error("deal not found");
```

```
  const acts = await db.query.activities.findMany({
    where: eq(activities.dealId, dealId),
    orderBy: desc(activities.occurredAt),
    limit: 20,
  });

  const system = `You are a deal forecasting engine. Output JSON:
{"probability": 0-100, "reasoning": "...", "risks": ["..."], "nextActions": ["...
  const user = JSON.stringify({ deal: d, activities: acts });
  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: user }],
    jsonMode: true,
    maxTokens: 1500,
  });

  const parsed = res.json as {
    probability: number; reasoning: string; risks: string[]; nextActions: string[
  };

  await db.update(deals).set({
    aiForecast: parsed,
    probability: parsed.probability,
  }).where(eq(deals.id, dealId));

  return parsed;
}
```

packages/ai/src/agents/daily-briefing.ts

```
import { generate } from "../gateway";
import { db, contacts, signals, activities, deals } from "@nexflow/db";
import { and, eq, gte, desc } from "drizzle-orm";

export async function generateDailyBriefing(userId: string, orgId: string) {
  const since = new Date(Date.now() - 24 * 60 * 60 * 1000);
  const mySignals = await db.query.signals.findMany({
    where: and(eq(signals.orgId, orgId), gte(signals.createdAt, since)),
    orderBy: desc(signals.aiScore), limit: 10,
  });
  const openDeals = await db.query.deals.findMany({
    where: and(eq(deals.orgId, orgId), eq(deals.ownerUserId, userId)),
    orderBy: desc(deals.updatedAt), limit: 20,
  });
```

```
  const system = `Write a morning briefing for a sales rep in under 200 words.
Sections: (1) Top priorities for today, (2) Hot signals to act on, (3) Deals at r
  const user = JSON.stringify({ signals: mySignals, deals: openDeals });
  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: user }],
    maxTokens: 800,
  });
  return res.text;
}

export async function generateContactBrief({ contactId, orgId }: { contactId: str
  const c = await db.query.contacts.findFirst({
    where: and(eq(contacts.id, contactId), eq(contacts.orgId, orgId)),
    with: { company: true },
  });
  if (!c) throw new Error("not_found");

  const system = `Write a 150-word pre-call brief for a B2B sales rep. Sections:
Who they are · What they care about · Best opening line · Likely objection · Reco
  const user = JSON.stringify(c);
  const res = await generate({
    tier: "default",
    system,
    messages: [{ role: "user", content: user }],
    maxTokens: 800,
  });
  return { brief: res.text, tokensIn: res.tokensIn, tokensOut: res.tokensOut };
}
```

packages/ai/src/agents/ai-assistant.ts

```
import { generate, generateStream } from "../gateway";
import type { ModelTier } from "../gateway";

export async function* streamAssistant({
  orgId, userId, contextType, contextId, messages, tier = "default",
}: {
  orgId: string; userId: string; contextType?: string; contextId?: string;
  messages: Array<{ role: "user" | "assistant"; content: string }>;
  tier?: ModelTier;
}) {
  const system = `You are the NexFlow AI Assistant. You have access to the user's
Be concise. Prefer action over chatter. When the user asks for data, respond in s
If they ask to do something (send email, create deal), describe the action, then
```

```
<<TOOL: {"name":"create_deal","args":{...}}>>`;
  for await (const chunk of generateStream({ tier, system, messages, maxTokens: 4
    yield chunk;
  }
}
```

packages/ai/src/agents/contact-enricher.ts

```
import { db, contacts, companies } from "@nexflow/db";
import { eq } from "drizzle-orm";
import { enrichContactWaterfall } from "../../../apps/api/src/enrichment/waterfal

export async function enrichContact(contactId: string, orgId: string) {
  const c = await db.query.contacts.findFirst({ where: eq(contacts.id, contactId)
  if (!c) throw new Error("contact_not_found");

  const enriched = await enrichContactWaterfall({
    fullName: c.fullName, email: c.email ?? undefined,
    linkedinUrl: c.linkedinUrl ?? undefined, phone: c.phone ?? undefined,
    companyDomain: c.companyId ? undefined : undefined,
  });

  await db.update(contacts).set({
    title: enriched.title ?? c.title,
    phone: enriched.phone ?? c.phone,
    linkedinUrl: enriched.linkedinUrl ?? c.linkedinUrl,
    country: enriched.country ?? c.country,
    enrichmentSignals: enriched.signals,
    enrichmentConfidence: enriched.confidence,
    enrichmentLastRunAt: new Date(),
  }).where(eq(contacts.id, contactId));

  return enriched;
}
```

packages/ai/src/index.ts

```
export * from "./gateway";
export * from "./agents/signal-scorer";
export * from "./agents/lead-scorer";
export * from "./agents/compliance-screener";
export * from "./agents/call-coach";
export * from "./agents/call-scorer";
export * from "./agents/segment-generator";
```

```
export * from "./agents/deal-forecaster";
export * from "./agents/daily-briefing";
export * from "./agents/ai-assistant";
export * from "./agents/contact-enricher";
```

---

10. Compliance Screening Engine (complete)

`apps/api/src/compliance/engine.ts` — pluggable

```
import fetch from "node-fetch";
import type { ComplianceList, ComplianceMatch, ComplianceDecision } from "@nexflo

export interface ScreeningInput {
  fullName: string;
  dateOfBirth?: string;
  country?: string;
  aliases?: string[];
}

export interface ScreeningResult {
  decision: ComplianceDecision;      // clear | flagged | blocked
  matches: ComplianceMatch[];
  listsUsed: ComplianceList[];
  raw: Record<string, unknown>;
}

type Adapter = (input: ScreeningInput) => Promise<ComplianceMatch[]>;

const adapters: Record<ComplianceList, Adapter> = {
  ofac_sdn: ofacAdapter,
  un_consolidated: unAdapter,
  eu_sanctions: euAdapter,
  dj_pep: djPepAdapter,
  sama: samaAdapter,
  simah: simahAdapter,
  moj: mojAdapter,
  hmt_uk: hmtAdapter,
};

async function ofacAdapter(input: ScreeningInput): Promise<ComplianceMatch[]> {
  const url = `https://sanctionssearch.ofac.treas.gov/API/PublicationPreview/expo
  // Implementation note: in production, mirror the OFAC SDN file daily into Post
  // and run fuzzy matching (pg_trgm + levenshtein). This function does a name-ma
```

```
  const { data, raw } = await namedMatchFromLocalMirror(input, "ofac_sdn");
  return data;
}

async function unAdapter(i: ScreeningInput) { return namedMatchFromLocalMirror(i,
async function euAdapter(i: ScreeningInput) { return namedMatchFromLocalMirror(i,
async function djPepAdapter(i: ScreeningInput) { return vendorLookup(i, "dj_pep")
async function samaAdapter(i: ScreeningInput) { return vendorLookup(i, "sama"); }
async function simahAdapter(i: ScreeningInput) { return vendorLookup(i, "simah");
async function mojAdapter(i: ScreeningInput) { return vendorLookup(i, "moj"); }
async function hmtAdapter(i: ScreeningInput) { return vendorLookup(i, "hmt_uk");

async function namedMatchFromLocalMirror(
  input: ScreeningInput, list: ComplianceList
): Promise<{ data: ComplianceMatch[]; raw: unknown }> {
  // Replit AI: implement via Postgres function `match_name_in_list(list TEXT, na
  // using pg_trgm similarity. Return matches ≥ 0.65 similarity.
  return { data: [], raw: null };
}

async function vendorLookup(input: ScreeningInput, list: ComplianceList): Promise
  const keyMap: Record<string, string | undefined> = {
    dj_pep: process.env.DOW_JONES_API_KEY,
    sama: undefined, simah: undefined, moj: undefined, hmt_uk: undefined,
  };
  const apiKey = keyMap[list];
  if (!apiKey) return [];
  // Replit AI: plug Dow Jones / ComplyAdvantage / Refinitiv endpoint here per co
  return [];
}

export async function runScreening(
  input: ScreeningInput,
  opts: { orgId: string; lists?: ComplianceList[] }
): Promise<ScreeningResult> {
  const listsToUse = opts.lists ?? await listsForOrg(opts.orgId);
  const results = await Promise.all(listsToUse.map((l) => adapters[l](input)));
  const matches = results.flat();

  const maxScore = matches.reduce((m, x) => Math.max(m, x.score), 0);
  const decision: ComplianceDecision =
    maxScore >= 90 ? "blocked" :
    maxScore >= 65 ? "flagged" : "clear";

  return { decision, matches, listsUsed: listsToUse, raw: { runAt: new Date().toI
}
```

```
async function listsForOrg(orgId: string): Promise<ComplianceList[]> {
  // Replit AI: read from organizations.settings.complianceLists
  return ["ofac_sdn", "un_consolidated", "eu_sanctions", "dj_pep"];
}
```

`packages/shared/src/types.ts` — compliance types

```
export type ComplianceList =
  | "ofac_sdn" | "un_consolidated" | "eu_sanctions" | "dj_pep"
  | "sama" | "simah" | "moj" | "hmt_uk";

export type ComplianceDecision = "clear" | "flagged" | "blocked";

export interface ComplianceMatch {
  list: ComplianceList;
  score: number;               // 0-100 similarity
  matchedName: string;
  matchedFields: string[];
  sourceUrl?: string;
}
```

---

11. Enrichment Waterfall (complete)

`apps/api/src/enrichment/waterfall.ts`

```
export interface EnrichmentInput {
  fullName?: string;
  email?: string;
  phone?: string;
  linkedinUrl?: string;
  companyDomain?: string;
}

export interface EnrichedContact {
  title?: string;
  phone?: string;
  linkedinUrl?: string;
  country?: string;
  signals: Record<string, unknown>;
  confidence: "low" | "medium" | "high";
}
```

```
// Providers try in order; first hit with ≥ medium confidence wins.
const PROVIDER_ORDER = ["peopledatalabs", "apollo", "clearbit", "hunter"] as cons

export async function enrichContactWaterfall(input: EnrichmentInput): Promise<Enr
  const signals: Record<string, unknown> = {};
  let title: string | undefined, phone: string | undefined,
      linkedinUrl: string | undefined, country: string | undefined;
  let confidence: "low" | "medium" | "high" = "low";

  for (const p of PROVIDER_ORDER) {
    const r = await callProvider(p, input);
    if (!r) continue;
    signals[p] = r;
    title ||= r.title; phone ||= r.phone;
    linkedinUrl ||= r.linkedinUrl; country ||= r.country;
    if (r.confidence === "high") { confidence = "high"; break; }
    if (r.confidence === "medium" && confidence === "low") confidence = "medium";
  }
  return { title, phone, linkedinUrl, country, signals, confidence };
}

async function callProvider(p: string, input: EnrichmentInput) {
  switch (p) {
    case "peopledatalabs": return pdl(input);
    case "apollo": return apollo(input);
    case "clearbit": return clearbit(input);
    case "hunter": return hunter(input);
    default: return null;
  }
}

async function pdl(input: EnrichmentInput) {
  const key = process.env.PEOPLEDATALABS_API_KEY;
  if (!key || !input.email) return null;
  const res = await fetch(`https://api.peopledatalabs.com/v5/person/enrich?email=
    headers: { "X-Api-Key": key },
  });
  if (!res.ok) return null;
  const j = await res.json() as any;
  return {
    title: j.data?.job_title, phone: j.data?.mobile_phone,
    linkedinUrl: j.data?.linkedin_url, country: j.data?.location_country,
    confidence: (j.likelihood ?? 0) >= 8 ? "high" : "medium" as const,
  };
}

async function apollo(input: EnrichmentInput) {
```

```
  const key = process.env.APOLLO_API_KEY;
  if (!key || !input.email) return null;
  const res = await fetch(`https://api.apollo.io/api/v1/people/match`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "X-Api-Key": key },
    body: JSON.stringify({ email: input.email }),
  });
  if (!res.ok) return null;
  const j = await res.json() as any;
  return {
    title: j.person?.title, phone: j.person?.phone_numbers?.[0]?.sanitized_number
    linkedinUrl: j.person?.linkedin_url, country: j.person?.country,
    confidence: "medium" as const,
  };
}

async function clearbit(input: EnrichmentInput) {
  const key = process.env.CLEARBIT_API_KEY;
  if (!key || !input.email) return null;
  const res = await fetch(`https://person.clearbit.com/v2/people/find?email=${enc
    headers: { Authorization: `Bearer ${key}` },
  });
  if (!res.ok) return null;
  const j = await res.json() as any;
  return {
    title: j.employment?.title, phone: j.phone,
    linkedinUrl: j.linkedin?.handle ? `https://linkedin.com/in/${j.linkedin.handl
    country: j.geo?.country,
    confidence: "medium" as const,
  };
}

async function hunter(input: EnrichmentInput) {
  const key = process.env.HUNTER_API_KEY;
  if (!key || !input.email) return null;
  const res = await fetch(`https://api.hunter.io/v2/people/find?email=${encodeURI
  if (!res.ok) return null;
  const j = await res.json() as any;
  return {
    title: j.data?.position, phone: j.data?.phone_number,
    linkedinUrl: j.data?.linkedin, country: j.data?.country,
    confidence: "low" as const,
  };
}
```

12. Voice Stack — Realtime Calls (complete)

Flow

1. Rep clicks "Call" on a contact → `POST /v1/calls/start` with `{ contactId, voiceAgentId?, script? }`.
2. API creates a `calls` row (status=queued), originates a Twilio call with `<Stream>` TwiML pointing to the WS endpoint.
3. Twilio media stream hits `WS /ws/calls/:id/stream`.
4. API pipes inbound audio → Whisper (streaming transcription).
5. Each transcript chunk → Claude Sonnet (call-coach) → whisper suggestion pushed to rep UI via `WS /ws/calls/:id/client`.
6. Call ends → `POST /v1/calls/:id/end` stores transcript, triggers `scoreCompletedCall`.

`apps/api/src/routes/calls.ts` — key handler

```
import { Hono } from "hono";
import { zValidator } from "@hono/zod-validator";
import { z } from "zod";
import twilio from "twilio";
import { db, calls, activities } from "@nexflow/db";
import { and, eq } from "drizzle-orm";
import { scoreCompletedCall } from "@nexflow/ai";

const app = new Hono();
const tw = twilio(process.env.TWILIO_ACCOUNT_SID, process.env.TWILIO_AUTH_TOKEN);

app.post("/start", zValidator("json", z.object({
  contactId: z.string().uuid(),
  toNumber: z.string(),
  voiceAgentId: z.string().uuid().optional(),
})), async (c) => {
  const { contactId, toNumber, voiceAgentId } = c.req.valid("json");
  const orgId = c.get("orgId") as string;
  const user = c.get("user") as { id: string };

  const [call] = await db.insert(calls).values({
    orgId, contactId, userId: user.id, voiceAgentId,
    status: "queued", direction: "outbound",
    fromNumber: process.env.TWILIO_PHONE_NUMBER!, toNumber,
  }).returning();
```

```
  const twiml = `
<Response>
  <Start><Stream url="${process.env.WS_URL}/ws/calls/${call.id}/stream"/></Start>
  <Dial callerId="${process.env.TWILIO_PHONE_NUMBER}">${toNumber}</Dial>
</Response>`.trim();

  const twilioCall = await tw.calls.create({
    twiml, to: toNumber, from: process.env.TWILIO_PHONE_NUMBER!,
    statusCallback: `${process.env.API_URL}/v1/calls/twilio-status`,
    statusCallbackEvent: ["initiated", "ringing", "answered", "completed"],
  });

  await db.update(calls).set({ twilioCallSid: twilioCall.sid }).where(eq(calls.id
  return c.json({ callId: call.id, twilioSid: twilioCall.sid });
});

app.post("/:id/end", async (c) => {
  const id = c.req.param("id");
  const orgId = c.get("orgId") as string;
  const row = await db.query.calls.findFirst({ where: and(eq(calls.id, id), eq(ca
  if (!row) return c.json({ error: "not_found" }, 404);

  await db.update(calls).set({ status: "completed", endedAt: new Date() }).where(
  const scored = await scoreCompletedCall(id);
  return c.json(scored);
});

export default app;
```

apps/api/src/ws/call-room.ts

```
import { WebSocketServer, WebSocket } from "ws";
import OpenAI from "openai";
import { suggestCoachingPrompt } from "@nexflow/ai";

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! });

export function mountCallWs(wss: WebSocketServer) {
  wss.on("connection", (ws: WebSocket, req) => {
    const url = new URL(req.url!, "ws://x");
    const m = url.pathname.match(/\/ws\/calls\/([^/]+)\/(stream|client)/);
    if (!m) return ws.close(1008, "bad path");
    const [, callId, kind] = m;

    if (kind === "stream") {
```

```
      // Buffer Twilio media, transcribe via Whisper every ~2s, relay to client
      const buffer: Buffer[] = [];
      ws.on("message", async (raw) => {
        const msg = JSON.parse(raw.toString());
        if (msg.event === "media") buffer.push(Buffer.from(msg.media.payload, "ba
        if (buffer.length >= 20) {
          const audio = Buffer.concat(buffer.splice(0));
          const transcription = await openai.audio.transcriptions.create({
            file: new File([audio], "chunk.wav", { type: "audio/wav" }),
            model: "whisper-1",
          });
          const hint = await suggestCoachingPrompt({
            transcript: [{ speaker: "contact", text: transcription.text }],
            contactContext: "", objective: "book meeting",
          });
          // TODO: persist transcript chunk + push via pub/sub to client WS
        }
      });
    }
  });
}
```

---

13. WhatsApp / Email / SMS Flows

apps/api/src/routes/whatsapp.ts

```
import { Hono } from "hono";
import { z } from "zod";
import { zValidator } from "@hono/zod-validator";
import fetch from "node-fetch";

const app = new Hono();

// Meta webhook verification
app.get("/webhook", (c) => {
  const mode = c.req.query("hub.mode");
  const token = c.req.query("hub.verify_token");
  const challenge = c.req.query("hub.challenge");
  if (mode === "subscribe" && token === process.env.WHATSAPP_VERIFY_TOKEN) {
    return c.text(challenge ?? "");
  }
  return c.text("forbidden", 403);
});
```

```
app.post("/webhook", async (c) => {
  const body = await c.req.json();
  // enqueue inbound message job
  await import("../jobs/enqueue").then((m) => m.enqueueWhatsappInbound(body));
  return c.text("OK");
});

app.post("/send", zValidator("json", z.object({
  to: z.string(), template: z.string(), variables: z.record(z.string()).optional(
  body: z.string().optional(),
})), async (c) => {
  const { to, template, variables, body } = c.req.valid("json");
  const res = await fetch(`https://graph.facebook.com/v21.0/${process.env.WHATSAP
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.WHATSAPP_ACCESS_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      messaging_product: "whatsapp", to,
      type: template ? "template" : "text",
      template: template ? {
        name: template,
        language: { code: "en_US" },
        components: variables ? [{
          type: "body",
          parameters: Object.values(variables).map((v) => ({ type: "text", text:
        }] : undefined,
      } : undefined,
      text: body ? { body } : undefined,
    }),
  });
  const j = await res.json();
  return c.json(j);
});

export default app;
```

---

14. Frontend — Next.js 15 App (complete)

apps/web/package.json

```
{
  "name": "@nexflow/web",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "next dev -p 3000",
    "build": "next build",
    "start": "next start -p 3000",
    "lint": "next lint",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@clerk/nextjs": "^6.5.0",
    "@hookform/resolvers": "^3.9.0",
    "@nexflow/shared": "workspace:*",
    "@nexflow/ui": "workspace:*",
    "@radix-ui/react-dialog": "^1.1.2",
    "@radix-ui/react-dropdown-menu": "^2.1.2",
    "@radix-ui/react-slot": "^1.1.0",
    "@radix-ui/react-tabs": "^1.1.1",
    "@radix-ui/react-toast": "^1.2.2",
    "@tanstack/react-query": "^5.59.0",
    "class-variance-authority": "^0.7.0",
    "clsx": "^2.1.1",
    "date-fns": "^4.1.0",
    "framer-motion": "^11.11.0",
    "lucide-react": "^0.450.0",
    "next": "^15.0.3",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-hook-form": "^7.53.0",
    "recharts": "^2.13.0",
    "tailwind-merge": "^2.5.4",
    "tailwindcss-animate": "^1.0.7",
    "zod": "^3.23.8",
    "zustand": "^5.0.0"
  },
  "devDependencies": {
    "@types/node": "^22.9.0",
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1",
    "autoprefixer": "^10.4.20",
    "eslint": "^9.14.0",
    "eslint-config-next": "^15.0.3",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.14",
```

```
    "typescript": "^5.6.3"
  }
}
```

apps/web/next.config.mjs

```
const nextConfig = {
  reactStrictMode: true,
  transpilePackages: ["@nexflow/ui", "@nexflow/shared"],
  experimental: { serverComponentsExternalPackages: ["postgres"] },
  async rewrites() {
    return [
      { source: "/api/v1/:path*", destination: `${process.env.API_URL}/v1/:path*`
    ];
  },
};
export default nextConfig;
```

apps/web/tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: "class",
  content: ["./app/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "../../packages/
  theme: {
    extend: {
      colors: {
        border: "hsl(var(--border))",
        background: "hsl(var(--background))",
        foreground: "hsl(var(--foreground))",
        primary: { DEFAULT: "hsl(var(--primary))", foreground: "hsl(var(--primary
        accent: { DEFAULT: "hsl(var(--accent))", foreground: "hsl(var(--accent-fo
        muted: { DEFAULT: "hsl(var(--muted))", foreground: "hsl(var(--muted-foreg
        card: { DEFAULT: "hsl(var(--card))", foreground: "hsl(var(--card-foregrou
        nf: {
          teal: "#00c896", pink: "#ff6b6b", violet: "#8b5cf6",
          lavender: "#B8A0C8", mesh1: "#4b0082", mesh2: "#1a0033",
        },
      },
      fontFamily: {
        sans: ["DM Sans", "system-ui"],
        display: ["DM Serif Display", "serif"],
        mono: ["DM Mono", "monospace"],
```

```
        arabic: ["IBM Plex Sans Arabic", "sans-serif"],
      },
    },
  },
  plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

apps/web/app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 240 10% 4%;
    --foreground: 0 0% 98%;
    --card: 240 10% 6%;
    --card-foreground: 0 0% 98%;
    --muted: 240 4% 16%;
    --muted-foreground: 240 5% 64%;
    --primary: 165 100% 39%;
    --primary-foreground: 0 0% 100%;
    --accent: 280 60% 65%;
    --accent-foreground: 0 0% 100%;
    --border: 240 4% 16%;
  }
}

.nf-mesh {
  background:
    radial-gradient(at 20% 20%, rgba(184,160,200,.25) 0px, transparent 40%),
    radial-gradient(at 80% 10%, rgba(0,200,150,.18) 0px, transparent 40%),
    radial-gradient(at 50% 90%, rgba(255,107,107,.15) 0px, transparent 40%),
    #0a0816;
}

.glass {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.06);
  backdrop-filter: blur(24px);
}
```

apps/web/app/(app)/layout.tsx

```
import { ClerkProvider } from "@clerk/nextjs";
import { Sidebar } from "@/components/sidebar";
import { TopBar } from "@/components/top-bar";
import { QueryProvider } from "@/lib/query-provider";

export default function AppLayout({ children }: { children: React.ReactNode }) {
  return (
    <ClerkProvider>
      <QueryProvider>
        <div className="min-h-screen nf-mesh text-foreground">
          <div className="flex h-screen">
            <Sidebar />
            <div className="flex flex-1 flex-col overflow-hidden">
              <TopBar />
              <main className="flex-1 overflow-auto p-6">{children}</main>
            </div>
          </div>
        </div>
      </QueryProvider>
    </ClerkProvider>
  );
}
```

apps/web/components/sidebar.tsx

```
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { Home, Users, Building2, TrendingUp, Phone, MessageSquare, Zap, FileText,

const modules = [
  { href: "/", icon: Home, label: "Home" },
  { href: "/dashboard", icon: BarChart3, label: "Dashboard" },
  { href: "/contacts", icon: Users, label: "Contacts" },
  { href: "/companies", icon: Building2, label: "Companies" },
  { href: "/signals", icon: Radar, label: "Signals" },
  { href: "/harvest", icon: Search, label: "Harvest" },
  { href: "/segments", icon: Zap, label: "Segments" },
  { href: "/calls", icon: Phone, label: "Calls" },
  { href: "/voice-studio", icon: Sparkles, label: "Voice Studio" },
  { href: "/whatsapp", icon: MessageSquare, label: "WhatsApp" },
  { href: "/scripts", icon: FileText, label: "Scripts" },
  { href: "/deals", icon: Briefcase, label: "Pipeline" },
  { href: "/assistant", icon: Sparkles, label: "Assistant" },
  { href: "/notifications", icon: Bell, label: "Notifications" },
```

```
  { href: "/forecast", icon: TrendingUp, label: "Forecast" },
  { href: "/calendar", icon: Calendar, label: "Calendar" },
  { href: "/analytics", icon: BarChart3, label: "Analytics" },
  { href: "/admin", icon: Settings, label: "Admin" },
];

export function Sidebar() {
  const pathname = usePathname();
  return (
    <aside className="w-60 glass p-4 flex flex-col gap-1">
      <div className="flex items-center gap-2 pb-4 mb-2 border-b border-white/5">
        <div className="h-8 w-8 rounded-lg bg-gradient-to-br from-nf-teal to-nf-v
        <span className="font-display text-xl tracking-tight">NexFlow</span>
      </div>
      {modules.map((m) => {
        const active = pathname === m.href || (m.href !== "/" && pathname.startsW
        return (
          <Link key={m.href} href={m.href}
            className={`flex items-center gap-3 rounded-md px-3 py-2 text-sm tran
              active ? "bg-white/5 text-white" : "text-muted-foreground hover:bg-
            }`}>
            <m.icon className="h-4 w-4" />
            <span>{m.label}</span>
          </Link>
        );
      })}
    </aside>
  );
}
```

apps/web/app/(app)/contacts/page.tsx

```
"use client";
import { useQuery } from "@tanstack/react-query";
import { ContactTable } from "@/components/contact-table";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Plus } from "lucide-react";
import { useState } from "react";
import { api } from "@/lib/api";

export default function ContactsPage() {
  const [search, setSearch] = useState("");
  const { data, isLoading } = useQuery({
    queryKey: ["contacts", search],
    queryFn: () => api.get(`/v1/contacts?search=${encodeURIComponent(search)}`),
```

```
  });
  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="font-display text-3xl">Contacts</h1>
          <p className="text-muted-foreground text-sm">
            {data?.items?.length ?? 0} shown
          </p>
        </div>
        <Button><Plus className="h-4 w-4 mr-2" />Add contact</Button>
      </div>
      <Input placeholder="Search contacts…" value={search} onChange={(e) => setSe
      <ContactTable contacts={data?.items ?? []} loading={isLoading} />
    </div>
  );
}
```

apps/web/lib/api.ts

```
import { auth } from "@clerk/nextjs/server";

async function request<T>(method: string, path: string, body?: unknown): Promise<
  const { getToken } = auth();
  const token = await getToken();
  const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}${path}`, {
    method,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
    cache: "no-store",
  });
  if (!res.ok) throw new Error(`${res.status}: ${await res.text()}`);
  return res.json();
}

export const api = {
  get: <T = any>(p: string) => request<T>("GET", p),
  post: <T = any>(p: string, b?: unknown) => request<T>("POST", p, b),
  patch: <T = any>(p: string, b?: unknown) => request<T>("PATCH", p, b),
  del: <T = any>(p: string) => request<T>("DELETE", p),
};
```

The remaining 16 module pages (home, dashboard, contacts/[id], companies, signals, harvest, segments, calls, voice-studio, whatsapp, scripts, deals, assistant, notifications, forecast, calendar, analytics, admin) follow identical patterns — a client component with React Query fetching `/v1/{resource}`, rendering with shadcn primitives. **Replit AI must produce one file per module using the contacts page pattern above as the template.**

14b. NexFlow Full Blend Brand Identity (complete)

This is the **NexFlow Full Blend design system** — the visual language the app must render in. Replit AI must apply every element below exactly. Do not invent colors, rename tokens, or drop animations. Do not substitute "generic dark theme" or "default shadcn".

14b.1 Design Principles

The Full Blend system has four governing rules:

1. **Chameleon Palette** — the brand is a 6-color DNA that shifts continuously through gradient animations. No flat solid brand color. The logo, wordmark, primary buttons, tab underlines, and section accents all cycle through the DNA on a timed loop.
2. **Living Mesh** — the background is never flat. It's a composition of 10 radial-gradient nodes in the Chameleon colors, each drifting along a unique path so no two moments look identical.
3. **Pulse Rule** — key brand marks (logo, AI assistant avatar, status dots) breathe at 2.4s on a `scale(1.0 → 1.05 → 1.0)` heartbeat.
4. **Flow Rule** — line art (logo diamonds, dividers, decorative strokes) is drawn in via `stroke-dashoffset` animation on load — the brand is "drawn by the AI in real-time."

14b.2 Chameleon Palette (DNA)

Token	Hex	Role
`--nf-primary`	`#B8A0C8`	Soft Muted Lavender — primary brand
`--nf-secondary`	`#C0A0B8`	Warm Rose-Mauve — secondary brand
`--nf-growth`	`#88B8B0`	Desaturated Seafoam — growth / positive
`--nf-wealth`	`#C8A880`	Muted Sand / Amber — wealth / value

`--nf-interface`	`#90B8B8`	Mist Green-Blue — interface surfaces
`--nf-highlight`	`#B8B880`	Muted Olive-Lemon — highlights
`--nf-base`	`#F5F2F6`	Base canvas (lavender-tinted cream)
`--nf-base-2`	`#EDE7F0`	Soft lavender haze
`--nf-base-3`	`#E0D6E5`	Deeper lavender tint
`--nf-ink`	`#3E2F4A`	Deep Mauve-Violet (primary text)
`--nf-ink-2`	`#564263`	Mid mauve-violet
`--nf-ink-3`	`#8B7A95`	Secondary text
`--nf-ink-4`	`#B8A9C2`	Tertiary / placeholder
`--nf-line`	`rgba(62,47,74,0.08)`	Soft divider
`--nf-line-2`	`rgba(62,47,74,0.14)`	Medium divider
`--nf-paper`	`rgba(255,255,255,0.78)`	Glassmorphism surface

Status colors (harmonized to palette):

Token	Hex	Role
`--nf-status-growth`	`#5A9088`	Positive / success
`--nf-status-alert`	`#B07080`	Alert / error
`--nf-status-warn`	`#A89050`	Warning / caution

14b.3 Chameleon Gradient

The brand gradient is a 7-stop linear gradient sweeping the DNA:

```
--nf-chameleon: linear-gradient(90deg,
  var(--nf-primary)    0%,
  var(--nf-secondary) 18%,
  var(--nf-growth)    36%,
```

```
    var(--nf-interface) 54%,
    var(--nf-highlight) 72%,
    var(--nf-wealth)    90%,
    var(--nf-primary)  100%);
```

Applied to text via `background-clip: text` + `color: transparent`, with `background-size: 400%` and animated `background-position`.

14b.4 Animation Timings (governed)

Loop	Duration	Applies to
`--nf-logo-loop`	`6s`	Logo, wordmark (high-energy)
`--nf-button-loop`	`8s`	Primary buttons, tab underlines (interactive)
`--nf-ambient-loop`	`10s`	Tagline, ambient accents (subtle)
`--nf-pulse`	`2.4s`	Logo pulse, AI avatar pulse, live-status dots
Living Mesh node drifts	`22s–30s`	Background nodes (each node unique path)

14b.5 Typography

Font stack:

- **Display / headers:** `DM Serif Display` (elegant editorial feel) and `Fraunces` (italic cuts for avatars/brand wordmark)
- **Body / UI:** `DM Sans` and `Inter` (with feature settings `"cv11","ss01","ss03"`)
- **Mono / data:** `DM Mono` and `JetBrains Mono`
- **Arabic (RTL support):** `IBM Plex Sans Arabic`

Type scale (8px base unit):

Name	Size / line-height	Weight	Use
Display L	`48 / 56`	700	Page title
Display S	`36 / 44`	700	Section hero
H1	`30 / 38`	600	Primary heading
H2	`24 / 32`	600	Secondary heading

H3	`20 / 28`	600	Card heading
Body L	`18 / 28`	400	Intro paragraph
Body	`16 / 24`	400	Default body
Body S	`14 / 20`	400	Compact UI text
Caption	`12 / 18`	400	Labels, timestamps
Overline	`11 / 16`	600	Uppercase tags, `letter-spacing: 0.5px`

14b.6 The Logo (concentric diamond mark)

Three concentric diamonds (rotated squares) with a pulsing core dot. Each ring strokes in on load with staggered delays (Flow rule), then its stroke color cycles through the Chameleon palette (6s loop).

Full SVG — paste as `apps/web/components/nexflow-logo.tsx`:

```
"use client";
import * as React from "react";

export interface NexFlowLogoProps {
  size?: number;
  animated?: boolean;
  className?: string;
}

export function NexFlowLogo({ size = 40, animated = true, className }: NexFlowLog
  const id = React.useId();
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 220 220"
      className={className}
      aria-label="NexFlow"
      role="img"
    >
      <defs>
        <linearGradient id={`${id}-outer`} x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#B8A0C8">
            {animated && <animate attributeName="stop-color"
              values="#B8A0C8;#C0A0B8;#88B8B0;#90B8B8;#B8B880;#C8A880;#B8A0C8"
```

```
          dur="6s" repeatCount="indefinite" />}
      </stop>
      <stop offset="100%" stopColor="#C0A0B8">
        {animated && <animate attributeName="stop-color"
          values="#C0A0B8;#88B8B0;#90B8B8;#B8B880;#C8A880;#B8A0C8;#C0A0B8"
          dur="6s" repeatCount="indefinite" />}
      </stop>
    </linearGradient>
    <linearGradient id={`${id}-middle`} x1="0%" y1="0%" x2="100%" y2="100%">
      <stop offset="0%" stopColor="#88B8B0">
        {animated && <animate attributeName="stop-color"
          values="#88B8B0;#90B8B8;#B8B880;#C8A880;#B8A0C8;#C0A0B8;#88B8B0"
          dur="6s" repeatCount="indefinite" />}
      </stop>
      <stop offset="100%" stopColor="#90B8B8">
        {animated && <animate attributeName="stop-color"
          values="#90B8B8;#B8B880;#C8A880;#B8A0C8;#C0A0B8;#88B8B0;#90B8B8"
          dur="6s" repeatCount="indefinite" />}
      </stop>
    </linearGradient>
    <linearGradient id={`${id}-inner`} x1="0%" y1="0%" x2="100%" y2="100%">
      <stop offset="0%" stopColor="#C8A880">
        {animated && <animate attributeName="stop-color"
          values="#C8A880;#B8A0C8;#C0A0B8;#88B8B0;#90B8B8;#B8B880;#C8A880"
          dur="6s" repeatCount="indefinite" />}
      </stop>
      <stop offset="100%" stopColor="#B8B880">
        {animated && <animate attributeName="stop-color"
          values="#B8B880;#C8A880;#B8A0C8;#C0A0B8;#88B8B0;#90B8B8;#B8B880"
          dur="6s" repeatCount="indefinite" />}
      </stop>
    </linearGradient>
    <radialGradient id={`${id}-core`} cx="50%" cy="50%" r="50%">
      <stop offset="0%" stopColor="#B8A0C8" stopOpacity="0.95" />
      <stop offset="100%" stopColor="#C0A0B8" stopOpacity="0.75" />
    </radialGradient>
  </defs>

  <g style={animated ? { animation: "nf-pulse 2.4s ease-in-out infinite", tra
    <rect
      x="25" y="25" width="170" height="170" rx="6"
      transform="rotate(45 110 110)"
      fill="none" stroke={`url(#${id}-outer)`} strokeWidth="3.5"
      strokeLinejoin="round" strokeLinecap="round"
      style={animated ? { strokeDasharray: 400, strokeDashoffset: 400, animat
    />
    <rect
```

```
          x="55" y="55" width="110" height="110" rx="5"
          transform="rotate(45 110 110)"
          fill="none" stroke={`url(#${id}-middle)`} strokeWidth="3.5"
          strokeLinejoin="round" strokeLinecap="round"
          style={animated ? { strokeDasharray: 400, strokeDashoffset: 400, animat
        />
        <rect
          x="80" y="80" width="60" height="60" rx="4"
          transform="rotate(45 110 110)"
          fill="none" stroke={`url(#${id}-inner)`} strokeWidth="3.5"
          strokeLinejoin="round" strokeLinecap="round"
          style={animated ? { strokeDasharray: 400, strokeDashoffset: 400, animat
        />
        <circle cx="110" cy="110" r="10" fill={`url(#${id}-core)`} />
        <circle cx="110" cy="110" r="10" fill="none" stroke="#B8A0C8" strokeWidth
          {animated && (
            <>
              <animate attributeName="r" values="10;14;10" dur="2.4s" repeatCount
              <animate attributeName="stroke-opacity" values="0.4;0;0.4" dur="2.4
            </>
          )}
        </circle>
      </g>
    </svg>
  );
}

export function NexFlowLogoCompact({ size = 28 }: { size?: number }) {
  const id = React.useId();
  return (
    <svg width={size} height={size} viewBox="0 0 40 40" aria-label="NexFlow">
      <defs>
        <linearGradient id={`${id}-o`} x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#B8A0C8" />
          <stop offset="100%" stopColor="#C0A0B8" />
        </linearGradient>
        <linearGradient id={`${id}-m`} x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#88B8B0" />
          <stop offset="100%" stopColor="#90B8B8" />
        </linearGradient>
        <radialGradient id={`${id}-c`} cx="50%" cy="50%" r="50%">
          <stop offset="0%" stopColor="#B8A0C8" stopOpacity="1" />
          <stop offset="100%" stopColor="#C0A0B8" stopOpacity="0.85" />
        </radialGradient>
      </defs>
      <rect x="6.5" y="6.5" width="27" height="27" rx="1.2" transform="rotate(45
            fill="none" stroke={`url(#${id}-o)`} strokeWidth="2.2" strokeLinejoin
```

```
      <rect x="11" y="11" width="18" height="18" rx="1" transform="rotate(45 20 2
            fill="none" stroke={`url(#${id}-m)`} strokeWidth="2.2" strokeLinejoin
      <circle cx="20" cy="20" r="2.2" fill={`url(#${id}-c)`} />
    </svg>
  );
}
```

14b.7 Complete `globals.css` (replaces Section 14 stub)

`apps/web/app/globals.css` — full Full Blend stylesheet:

```
@tailwind base;
@tailwind components;
@tailwind utilities;

/* ═══════════════════════════════════════════════════════════
   NexFlow Full Blend — root tokens
   ═══════════════════════════════════════════════════════════ */
:root {
  /* Chameleon DNA */
  --nf-primary:    #B8A0C8;
  --nf-secondary:  #C0A0B8;
  --nf-growth:     #88B8B0;
  --nf-wealth:     #C8A880;
  --nf-interface:  #90B8B8;
  --nf-highlight:  #B8B880;

  /* Base surfaces */
  --nf-base:       #F5F2F6;
  --nf-base-2:     #EDE7F0;
  --nf-base-3:     #E0D6E5;
  --nf-paper:      rgba(255, 255, 255, 0.78);

  /* Ink (text) */
  --nf-ink:        #3E2F4A;
  --nf-ink-2:      #564263;
  --nf-ink-3:      #8B7A95;
  --nf-ink-4:      #B8A9C2;

  /* Lines */
  --nf-line:       rgba(62, 47, 74, 0.08);
  --nf-line-2:     rgba(62, 47, 74, 0.14);
  --nf-line-3:     rgba(62, 47, 74, 0.24);

  /* Status */
  --nf-status-growth:  #5A9088;
```

```
--nf-status-alert:   #B07080;
--nf-status-warn:    #A89050;

/* Chameleon gradient */
--nf-chameleon: linear-gradient(90deg,
  var(--nf-primary)    0%,
  var(--nf-secondary) 18%,
  var(--nf-growth)    36%,
  var(--nf-interface) 54%,
  var(--nf-highlight) 72%,
  var(--nf-wealth)    90%,
  var(--nf-primary)  100%);

/* Timings */
--nf-logo-loop:    6s;
--nf-button-loop:  8s;
--nf-ambient-loop: 10s;

/* Shadows */
--nf-shadow-sm:  0 2px 8px rgba(62, 47, 74, 0.06);
--nf-shadow-md:  0 4px 18px rgba(62, 47, 74, 0.10);
--nf-shadow-lg:  0 12px 40px rgba(62, 47, 74, 0.14);
--nf-shadow-glow: 0 4px 20px rgba(184, 160, 200, 0.35);

/* Radii */
--nf-r-xs:  4px;
--nf-r-sm:  8px;
--nf-r:    12px;
--nf-r-lg: 16px;
--nf-r-xl: 24px;

/* shadcn bridge — keep shadcn primitives compatible with Full Blend */
--background: 285 23% 96%;    /* --nf-base */
--foreground: 272 23% 24%;    /* --nf-ink */
--card: 0 0% 100%;
--card-foreground: 272 23% 24%;
--popover: 0 0% 100%;
--popover-foreground: 272 23% 24%;
--primary: 272 27% 71%;       /* --nf-primary */
--primary-foreground: 0 0% 100%;
--secondary: 318 22% 69%;     /* --nf-secondary */
--secondary-foreground: 272 23% 24%;
--muted: 283 17% 89%;
--muted-foreground: 281 10% 53%;
--accent: 171 22% 63%;        /* --nf-growth */
--accent-foreground: 272 23% 24%;
--destructive: 348 23% 57%;
```

```
  --destructive-foreground: 0 0% 100%;
  --border: 272 10% 85%;
  --input: 272 10% 85%;
  --ring: 272 27% 71%;
  --radius: 0.75rem;
}

/* Fonts */
@import url("https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;70

/* ═══════════════════════════════════════════════════════════
   Base typography
   ═══════════════════════════════════════════════════════════ */
@layer base {
  * { box-sizing: border-box; }
  html, body {
    font-family: "Inter", "DM Sans", system-ui, sans-serif;
    font-feature-settings: "cv11", "ss01", "ss03";
    -webkit-font-smoothing: antialiased;
    color: var(--nf-ink);
    background: var(--nf-base);
  }
  h1, h2, h3 { font-family: "DM Serif Display", "Fraunces", Georgia, serif; lette
  code, .mono { font-family: "DM Mono", "JetBrains Mono", monospace; }
  [dir="rtl"] { font-family: "IBM Plex Sans Arabic", "Inter", sans-serif; }
  ::selection { background: rgba(184, 160, 200, 0.35); color: var(--nf-ink); }
}

/* ═══════════════════════════════════════════════════════════
   LIVING MESH — background composition
   ═══════════════════════════════════════════════════════════ */
.nf-mesh {
  position: fixed;
  inset: 0;
  z-index: 0;
  overflow: hidden;
  pointer-events: none;
  background:
    linear-gradient(180deg, var(--nf-base) 0%, var(--nf-base-2) 100%);
}
.nf-mesh-node {
  position: absolute;
  width: 55vmax;
  height: 55vmax;
  border-radius: 50%;
  filter: blur(60px);
  will-change: transform, opacity;
```

```
}
.nf-mesh-node.n1  { background: radial-gradient(circle, var(--nf-primary)    0%,
.nf-mesh-node.n2  { background: radial-gradient(circle, var(--nf-growth)     0%,
.nf-mesh-node.n3  { background: radial-gradient(circle, var(--nf-wealth)     0%,
.nf-mesh-node.n4  { background: radial-gradient(circle, var(--nf-secondary)  0%,
.nf-mesh-node.n5  { background: radial-gradient(circle, var(--nf-interface)  0%,
.nf-mesh-node.n6  { background: radial-gradient(circle, var(--nf-highlight)  0%,
.nf-mesh-node.n7  { background: radial-gradient(circle, var(--nf-primary)    0%,
.nf-mesh-node.n8  { background: radial-gradient(circle, var(--nf-growth)     0%,
.nf-mesh-node.n9  { background: radial-gradient(circle, var(--nf-interface)  0%,
.nf-mesh-node.n10 { background: radial-gradient(circle, var(--nf-secondary)  0%,

@keyframes nf-f1  { 0%,100% { transform: translate(0,0) scale(1); }    50% { tran
@keyframes nf-f2  { 0%,100% { transform: translate(0,0) scale(1.1); }  50% { tran
@keyframes nf-f3  { 0%,100% { transform: translate(0,0) scale(1); }    50% { tran
@keyframes nf-f4  { 0%,100% { transform: translate(0,0) scale(1.05); } 50% { tran
@keyframes nf-f5  { 0%,100% { transform: translate(0,0) scale(1); }    50% { tran
@keyframes nf-f6  { 0%,100% { transform: translate(0,0) scale(1.1); }  50% { tran
@keyframes nf-f7  { 0%,100% { transform: translate(0,0) scale(1); }    50% { tran
@keyframes nf-f8  { 0%,100% { transform: translate(0,0) scale(1.05); } 50% { tran
@keyframes nf-f9  { 0%,100% { transform: translate(0,0) scale(1); }    50% { tran
@keyframes nf-f10 { 0%,100% { transform: translate(0,0) scale(1.1); }  50% { tran

/* ═══════════════════════════════════════════════════════════
   CHAMELEON — gradient text & button fills
   ═══════════════════════════════════════════════════════════ */
.nf-chameleon-text {
  background: var(--nf-chameleon);
  background-size: 400% 400%;
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
  animation: nf-shift var(--nf-logo-loop) ease-in-out infinite;
}
.nf-chameleon-text.ambient { animation-duration: var(--nf-ambient-loop); }
.nf-chameleon-fill {
  background: var(--nf-chameleon);
  background-size: 400% 400%;
  animation: nf-shift var(--nf-button-loop) ease-in-out infinite;
}
@keyframes nf-shift {
  0%, 100% { background-position: 0% 50%; }
  50%      { background-position: 100% 50%; }
}

/* ═══════════════════════════════════════════════════════════
   PULSE — logo heartbeat, AI avatar, status dots
```

```
   ═══════════════════════════════════════════════════════════ */
@keyframes nf-pulse {
  0%, 100% { transform: scale(1.00); }
  50%      { transform: scale(1.05); }
}
.nf-pulse { animation: nf-pulse 2.4s ease-in-out infinite; transform-origin: cent

/* ═══════════════════════════════════════════════════════════
   FLOW — stroke draw-in for line art
   ═══════════════════════════════════════════════════════════ */
@keyframes nf-draw {
  from { stroke-dashoffset: 400; }
  to   { stroke-dashoffset: 0;   }
}

/* ═══════════════════════════════════════════════════════════
   Glass — paper surface with backdrop blur
   ═══════════════════════════════════════════════════════════ */
.nf-glass {
  background: var(--nf-paper);
  border: 1px solid var(--nf-line);
  backdrop-filter: blur(18px) saturate(140%);
  -webkit-backdrop-filter: blur(18px) saturate(140%);
  box-shadow: var(--nf-shadow-md);
  border-radius: var(--nf-r);
}
.nf-glass-deep {
  background: rgba(255, 255, 255, 0.6);
  border: 1px solid var(--nf-line-2);
  backdrop-filter: blur(24px) saturate(160%);
  -webkit-backdrop-filter: blur(24px) saturate(160%);
  box-shadow: var(--nf-shadow-lg);
  border-radius: var(--nf-r-lg);
}

/* ═══════════════════════════════════════════════════════════
   Buttons
   ═══════════════════════════════════════════════════════════ */
.nf-btn {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 18px;
  border-radius: var(--nf-r-sm);
  font-family: "Inter", sans-serif;
  font-weight: 500;
  font-size: 14px;
```

```
  line-height: 1.2;
  cursor: pointer;
  transition: transform .15s ease, box-shadow .15s ease, filter .15s ease;
  border: 1px solid var(--nf-line-2);
  background: var(--nf-paper);
  color: var(--nf-ink);
}
.nf-btn:hover { transform: translateY(-1px); box-shadow: var(--nf-shadow-md); }
.nf-btn:active { transform: translateY(0); filter: brightness(0.97); }

.nf-btn-primary {
  color: #fff;
  border: none;
  background: var(--nf-chameleon);
  background-size: 400% 400%;
  animation: nf-shift var(--nf-button-loop) ease-in-out infinite;
  box-shadow: var(--nf-shadow-glow);
}
.nf-btn-primary:hover { filter: brightness(1.05); }

.nf-btn-ghost {
  background: transparent;
  border: 1px solid var(--nf-line-2);
}

/* ═══════════════════════════════════════════════════════════
   Inputs
   ═══════════════════════════════════════════════════════════ */
.nf-input {
  width: 100%;
  padding: 10px 14px;
  border-radius: var(--nf-r-sm);
  border: 1px solid var(--nf-line-2);
  background: rgba(255, 255, 255, 0.55);
  color: var(--nf-ink);
  font-family: "Inter", sans-serif;
  font-size: 14px;
  transition: border-color .15s ease, box-shadow .15s ease;
}
.nf-input:focus {
  outline: none;
  border-color: var(--nf-primary);
  box-shadow: 0 0 0 3px rgba(184, 160, 200, 0.25);
}
.nf-input::placeholder { color: var(--nf-ink-4); }

/* ═══════════════════════════════════════════════════════════
```

```
   Cards
   ═══════════════════════════════════════════════════════════ */
.nf-card {
  background: var(--nf-paper);
  border: 1px solid var(--nf-line);
  border-radius: var(--nf-r);
  padding: 20px;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  transition: box-shadow .2s ease, transform .2s ease;
}
.nf-card:hover { box-shadow: var(--nf-shadow-md); transform: translateY(-2px); }
.nf-card-title { font-family: "DM Serif Display", serif; font-size: 20px; margin:
.nf-card-sub { font-size: 13px; color: var(--nf-ink-3); }

/* ═══════════════════════════════════════════════════════════
   Tabs
   ═══════════════════════════════════════════════════════════ */
.nf-tabs { display: flex; gap: 4px; border-bottom: 1px solid var(--nf-line-2); }
.nf-tab {
  padding: 10px 16px; font-size: 14px; color: var(--nf-ink-3);
  cursor: pointer; position: relative; transition: color .15s ease;
}
.nf-tab:hover { color: var(--nf-ink); }
.nf-tab[data-active="true"] { color: var(--nf-ink); font-weight: 600; }
.nf-tab[data-active="true"]::after {
  content: ""; position: absolute; inset: auto 0 -1px 0; height: 2px; border-radi
  background: var(--nf-chameleon); background-size: 400% 400%;
  animation: nf-shift var(--nf-button-loop) ease-in-out infinite;
}

/* ═══════════════════════════════════════════════════════════
   Tables
   ═══════════════════════════════════════════════════════════ */
.nf-table { width: 100%; border-collapse: collapse; }
.nf-table th {
  text-align: left; font-size: 11px; font-weight: 600;
  letter-spacing: 0.5px; text-transform: uppercase;
  color: var(--nf-ink-3); padding: 12px 14px;
  border-bottom: 1px solid var(--nf-line-2);
}
.nf-table td {
  padding: 14px; font-size: 14px; color: var(--nf-ink-2);
  border-bottom: 1px solid var(--nf-line);
}
.nf-table tr:hover td { background: rgba(184, 160, 200, 0.06); }
```

```
/* ═══════════════════════════════════════════════════════════
   Badges / status dots
   ═══════════════════════════════════════════════════════════ */
.nf-badge {
  display: inline-flex; align-items: center; gap: 6px;
  padding: 3px 10px; border-radius: 999px;
  font-size: 11px; font-weight: 500; letter-spacing: 0.3px;
  background: rgba(184, 160, 200, 0.14); color: var(--nf-ink-2);
}
.nf-badge.growth { background: rgba(90, 144, 136, 0.16); color: var(--nf-status-g
.nf-badge.alert  { background: rgba(176, 112, 128, 0.16); color: var(--nf-status-
.nf-badge.warn   { background: rgba(168, 144, 80, 0.16);  color: var(--nf-status-

.nf-dot { width: 6px; height: 6px; border-radius: 50%; display: inline-block; }
.nf-dot.live { background: var(--nf-status-growth); animation: nf-pulse 2.4s ease

/* ═══════════════════════════════════════════════════════════
   AI Assistant avatar (orb)
   ═══════════════════════════════════════════════════════════ */
.nf-ai-orb {
  width: 38px; height: 38px; border-radius: 10px; flex-shrink: 0;
  background: linear-gradient(135deg, var(--nf-primary), var(--nf-secondary));
  color: #fff; font-family: "Fraunces", serif; font-style: italic;
  font-weight: 600; font-size: 19px;
  display: grid; place-items: center; letter-spacing: -1px;
  box-shadow: var(--nf-shadow-glow);
  animation: nf-pulse 2.4s ease-in-out infinite;
}

/* ═══════════════════════════════════════════════════════════
   Sidebar (brand rail)
   ═══════════════════════════════════════════════════════════ */
.nf-sidebar {
  width: 240px; height: 100vh; padding: 20px 14px;
  background: var(--nf-paper);
  border-right: 1px solid var(--nf-line);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  display: flex; flex-direction: column; gap: 2px;
  position: relative; z-index: 10;
}
.nf-sidebar-header {
  display: flex; align-items: center; gap: 10px;
  padding: 4px 8px 16px; border-bottom: 1px solid var(--nf-line);
  margin-bottom: 8px;
}
.nf-sidebar-wordmark {
```

```
  font-family: "DM Serif Display", serif;
  font-size: 20px; letter-spacing: -0.5px;
  background: var(--nf-chameleon); background-size: 400% 400%;
  -webkit-background-clip: text; background-clip: text; color: transparent;
  animation: nf-shift var(--nf-logo-loop) ease-in-out infinite;
}
.nf-sidebar-sub {
  font-family: "DM Mono", monospace; font-size: 9px;
  letter-spacing: 1.5px; color: var(--nf-ink-4);
  text-transform: uppercase; margin-top: 2px;
}
.nf-nav-item {
  display: flex; align-items: center; gap: 10px;
  padding: 9px 10px; border-radius: 8px;
  font-size: 13px; color: var(--nf-ink-3);
  cursor: pointer; transition: background .15s ease, color .15s ease;
}
.nf-nav-item:hover { background: rgba(184, 160, 200, 0.12); color: var(--nf-ink);
.nf-nav-item[data-active="true"] {
  background: rgba(184, 160, 200, 0.18);
  color: var(--nf-ink); font-weight: 500;
}
.nf-nav-item[data-active="true"] svg { color: var(--nf-primary); }

/* ═══════════════════════════════════════════════════════════
   Reduced-motion
   ═══════════════════════════════════════════════════════════ */
@media (prefers-reduced-motion: reduce) {
  .nf-mesh-node, .nf-chameleon-text, .nf-chameleon-fill,
  .nf-pulse, .nf-btn-primary, .nf-sidebar-wordmark,
  .nf-tab[data-active="true"]::after, .nf-ai-orb, .nf-dot.live {
    animation: none !important;
  }
}

/* ═══════════════════════════════════════════════════════════
   RTL support
   ═══════════════════════════════════════════════════════════ */
[dir="rtl"] .nf-sidebar { border-right: none; border-left: 1px solid var(--nf-lin
[dir="rtl"] .nf-tabs { direction: rtl; }
```

14b.8 Updated `tailwind.config.ts` (replaces Section 14 stub)

```
import type { Config } from "tailwindcss";

export default {
```

```
darkMode: ["class"],
content: ["./app/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "../../packages/
theme: {
  extend: {
    colors: {
      border: "hsl(var(--border))",
      input: "hsl(var(--input))",
      ring: "hsl(var(--ring))",
      background: "hsl(var(--background))",
      foreground: "hsl(var(--foreground))",
      primary: { DEFAULT: "hsl(var(--primary))", foreground: "hsl(var(--primary
      secondary: { DEFAULT: "hsl(var(--secondary))", foreground: "hsl(var(--sec
      accent: { DEFAULT: "hsl(var(--accent))", foreground: "hsl(var(--accent-fo
      muted: { DEFAULT: "hsl(var(--muted))", foreground: "hsl(var(--muted-foreg
      card: { DEFAULT: "hsl(var(--card))", foreground: "hsl(var(--card-foregrou
      popover: { DEFAULT: "hsl(var(--popover))", foreground: "hsl(var(--popover
      destructive: { DEFAULT: "hsl(var(--destructive))", foreground: "hsl(var(-
      // NexFlow DNA — direct access for custom components
      nf: {
        primary:   "#B8A0C8",
        secondary: "#C0A0B8",
        growth:    "#88B8B0",
        wealth:    "#C8A880",
        interface: "#90B8B8",
        highlight: "#B8B880",
        base:      "#F5F2F6",
        "base-2":  "#EDE7F0",
        "base-3":  "#E0D6E5",
        ink:       "#3E2F4A",
        "ink-2":   "#564263",
        "ink-3":   "#8B7A95",
        "ink-4":   "#B8A9C2",
      },
    },
    fontFamily: {
      sans:    ["Inter", "DM Sans", "system-ui", "sans-serif"],
      display: ["DM Serif Display", "Fraunces", "Georgia", "serif"],
      serif:   ["Fraunces", "Georgia", "serif"],
      mono:    ["DM Mono", "JetBrains Mono", "monospace"],
      arabic:  ["IBM Plex Sans Arabic", "Inter", "sans-serif"],
    },
    fontSize: {
      "display-l": ["48px", { lineHeight: "56px", fontWeight: "700" }],
      "display-s": ["36px", { lineHeight: "44px", fontWeight: "700" }],
      "h1":        ["30px", { lineHeight: "38px", fontWeight: "600" }],
      "h2":        ["24px", { lineHeight: "32px", fontWeight: "600" }],
      "h3":        ["20px", { lineHeight: "28px", fontWeight: "600" }],
```

```
        "body-l":    ["18px", { lineHeight: "28px", fontWeight: "400" }],
        "body":      ["16px", { lineHeight: "24px", fontWeight: "400" }],
        "body-s":    ["14px", { lineHeight: "20px", fontWeight: "400" }],
        "caption":   ["12px", { lineHeight: "18px", fontWeight: "400" }],
        "overline":  ["11px", { lineHeight: "16px", fontWeight: "600", letterSpac
      },
      borderRadius: {
        lg: "var(--radius)",
        md: "calc(var(--radius) - 2px)",
        sm: "calc(var(--radius) - 4px)",
      },
      boxShadow: {
        "nf-sm":   "0 2px 8px rgba(62, 47, 74, 0.06)",
        "nf":      "0 4px 18px rgba(62, 47, 74, 0.10)",
        "nf-lg":   "0 12px 40px rgba(62, 47, 74, 0.14)",
        "nf-glow": "0 4px 20px rgba(184, 160, 200, 0.35)",
      },
      animation: {
        "nf-pulse": "nf-pulse 2.4s ease-in-out infinite",
        "nf-shift": "nf-shift 6s ease-in-out infinite",
      },
    },
  },
  plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

14b.9 Living Mesh component

apps/web/components/living-mesh.tsx：

```
export function LivingMesh() {
  return (
    <div className="nf-mesh" aria-hidden>
      <div className="nf-mesh-node n1" />
      <div className="nf-mesh-node n2" />
      <div className="nf-mesh-node n3" />
      <div className="nf-mesh-node n4" />
      <div className="nf-mesh-node n5" />
      <div className="nf-mesh-node n6" />
      <div className="nf-mesh-node n7" />
      <div className="nf-mesh-node n8" />
      <div className="nf-mesh-node n9" />
      <div className="nf-mesh-node n10" />
    </div>
  );
}
```

14b.10 Updated `(app)/layout.tsx` (replaces Section 14 stub)

```
import { ClerkProvider } from "@clerk/nextjs";
import { LivingMesh } from "@/components/living-mesh";
import { Sidebar } from "@/components/sidebar";
import { TopBar } from "@/components/top-bar";
import { QueryProvider } from "@/lib/query-provider";

export default function AppLayout({ children }: { children: React.ReactNode }) {
  return (
    <ClerkProvider>
      <QueryProvider>
        <div className="relative min-h-screen">
          <LivingMesh />
          <div className="relative z-10 flex h-screen">
            <Sidebar />
            <div className="flex flex-1 flex-col overflow-hidden">
              <TopBar />
              <main className="flex-1 overflow-auto p-6">{children}</main>
            </div>
          </div>
        </div>
      </QueryProvider>
    </ClerkProvider>
  );
}
```

14b.11 Updated `Sidebar` (replaces Section 14 stub)

```
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  Home, Users, Building2, TrendingUp, Phone, MessageSquare, Zap,
  FileText, Briefcase, Bell, BarChart3, Calendar, Settings, Sparkles,
  Search, Radar, Mic, CircleUserRound,
} from "lucide-react";
import { NexFlowLogoCompact } from "./nexflow-logo";

const modules = [
  { href: "/",              icon: Home,          label: "Home" },
  { href: "/dashboard",     icon: BarChart3,     label: "Dashboard" },
  { href: "/contacts",      icon: Users,         label: "Contacts" },
  { href: "/companies",     icon: Building2,     label: "Companies" },
```

```
  { href: "/signals",        icon: Radar,          label: "Signals" },
  { href: "/harvest",        icon: Search,         label: "Harvest" },
  { href: "/segments",       icon: Zap,            label: "Segments" },
  { href: "/calls",          icon: Phone,          label: "Calls" },
  { href: "/voice-studio",   icon: Mic,            label: "Voice Studio" },
  { href: "/whatsapp",       icon: MessageSquare,  label: "WhatsApp" },
  { href: "/scripts",        icon: FileText,       label: "Scripts" },
  { href: "/deals",          icon: Briefcase,      label: "Pipeline" },
  { href: "/assistant",      icon: Sparkles,       label: "Assistant" },
  { href: "/notifications",  icon: Bell,           label: "Notifications" },
  { href: "/forecast",       icon: TrendingUp,     label: "Forecast" },
  { href: "/calendar",       icon: Calendar,       label: "Best-Time" },
  { href: "/analytics",      icon: BarChart3,      label: "Analytics" },
  { href: "/admin",          icon: Settings,       label: "Admin" },
];

export function Sidebar() {
  const pathname = usePathname();
  return (
    <aside className="nf-sidebar">
      <div className="nf-sidebar-header">
        <NexFlowLogoCompact size={32} />
        <div>
          <div className="nf-sidebar-wordmark">NexFlow</div>
          <div className="nf-sidebar-sub">SALES · INTEL · OS</div>
        </div>
      </div>
      <nav className="flex-1 overflow-y-auto">
        {modules.map((m) => {
          const active = pathname === m.href || (m.href !== "/" && pathname.start
          return (
            <Link
              key={m.href}
              href={m.href}
              className="nf-nav-item"
              data-active={active ? "true" : "false"}
            >
              <m.icon className="h-4 w-4" />
              <span>{m.label}</span>
            </Link>
          );
        })}
      </nav>
      <div className="border-t border-[var(--nf-line)] pt-3 mt-3">
        <Link href="/profile" className="nf-nav-item">
          <CircleUserRound className="h-4 w-4" />
          <span>Profile</span>
```

```
        </Link>
      </div>
    </aside>
  );
}
```

14b.12 TopBar with brand search + AI orb

`apps/web/components/top-bar.tsx`：

```
"use client";
import { Search, Bell } from "lucide-react";
import { UserButton } from "@clerk/nextjs";

export function TopBar() {
  return (
    <header className="nf-glass flex items-center justify-between gap-4 px-6 py-3
      <div className="flex items-center gap-3 flex-1 max-w-xl">
        <Search className="h-4 w-4 text-[var(--nf-ink-3)]" />
        <input
          className="nf-input bg-transparent border-0 shadow-none focus:shadow-no
          placeholder="Search contacts, companies, deals, signals…"
        />
      </div>
      <div className="flex items-center gap-3">
        <button className="nf-btn nf-btn-ghost" aria-label="Notifications">
          <Bell className="h-4 w-4" />
          <span className="nf-dot live" />
        </button>
        <div className="nf-ai-orb" aria-label="AI Assistant">N</div>
        <UserButton />
      </div>
    </header>
  );
}
```

14b.13 Updated `contacts/page.tsx` using Full Blend components

```
"use client";
import { useQuery } from "@tanstack/react-query";
import { Plus, Sparkles } from "lucide-react";
import { useState } from "react";
import { api } from "@/lib/api";

export default function ContactsPage() {
```

```
const [search, setSearch] = useState("");
const { data, isLoading } = useQuery({
  queryKey: ["contacts", search],
  queryFn: () => api.get(`/v1/contacts?search=${encodeURIComponent(search)}`),
});
return (
  <div className="space-y-5 max-w-7xl mx-auto">
    <div className="flex items-center justify-between">
      <div>
        <h1 className="nf-chameleon-text font-display text-display-s">Contacts<
        <p className="text-body-s text-[var(--nf-ink-3)] mt-1">
          {data?.items?.length ?? 0} shown · intelligent CRM core
        </p>
      </div>
      <div className="flex gap-2">
        <button className="nf-btn"><Sparkles className="h-4 w-4" /> Ask AI</but
        <button className="nf-btn nf-btn-primary"><Plus className="h-4 w-4" />
      </div>
    </div>

    <div className="nf-glass p-4">
      <input
        className="nf-input"
        placeholder="Search contacts by name, email, or company…"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
    </div>

    <div className="nf-glass overflow-hidden">
      <table className="nf-table">
        <thead>
          <tr>
            <th>Name</th>
            <th>Company</th>
            <th>Title</th>
            <th>Stage</th>
            <th>Score</th>
            <th>Compliance</th>
            <th>Owner</th>
          </tr>
        </thead>
        <tbody>
          {isLoading ? (
            <tr><td colSpan={7} className="text-center py-12 text-[var(--nf-ink
          ) : (data?.items ?? []).map((c: any) => (
            <tr key={c.id}>
```

```
                <td className="font-medium text-[var(--nf-ink)]">{c.fullName}</td
                <td>{c.company?.name ?? "—"}</td>
                <td>{c.title ?? "—"}</td>
                <td><span className="nf-badge">{c.lifecycleStage}</span></td>
                <td className="mono">{c.leadScore ?? 0}</td>
                <td>
                  <span className={`nf-badge ${
                    c.complianceStatus === "clear" ? "growth" :
                    c.complianceStatus === "flagged" ? "warn" :
                    c.complianceStatus === "blocked" ? "alert" : ""
                  }`}>
                    {c.complianceStatus}
                  </span>
                </td>
                <td>{c.owner?.fullName ?? "—"}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

14b.14 Brand usage rules

1. **Never use solid brand color for large surfaces.** The palette is gradient-first. Flat surfaces use `--nf-base` / `--nf-paper`.
2. **The Living Mesh is always on.** It sits in `z-index: 0` behind every authenticated screen.
3. **Chameleon applies only to brand marks, primary buttons, hero headers, and active tab underlines** — never to body text or long prose.
4. **Pulse is reserved for live / AI elements** — the logo, AI orb, live status dots. Never on static UI.
5. **Flow (stroke draw-in) applies only on first load of a brand mark** — does not repeat per render.
6. **Glass surfaces** (`.nf-glass` / `.nf-glass-deep`) are the default card / panel treatment. Flat cards are only for dense lists.
7. **RTL:** when `dir="rtl"` , auto-swap font stack to IBM Plex Sans Arabic and mirror sidebar border.

8. **Reduced-motion:** all looping animations stop. Static palette still applies.
9. **Do not introduce colors outside the DNA.** If you need a new hue for a chart series, sample it from the Chameleon palette only.
10. **Font fallbacks are mandatory** — never serve an unstyled page while Google Fonts load. The stack always starts with a system fallback.

14b.15 Component coverage checklist

Replit AI must apply `.nf-*` classes (or their Tailwind equivalents via the `nf` color tokens) to every one of these elements across every one of the 17 module pages:

- Page title → `nf-chameleon-text font-display text-display-s`
- Primary CTA → `nf-btn nf-btn-primary`
- Secondary button → `nf-btn`
- Ghost / tertiary → `nf-btn nf-btn-ghost`
- Input → `nf-input`
- Card / panel → `nf-glass` or `nf-glass-deep`
- Table → `nf-table`
- Tabs → `nf-tabs` + `nf-tab[data-active]`
- Badge → `nf-badge` + variant (`growth` / `alert` / `warn`)
- Live status dot → `nf-dot live`
- AI avatar → `nf-ai-orb`
- Sidebar → `nf-sidebar` + `nf-sidebar-wordmark` + `nf-nav-item`
- Logo → `<NexFlowLogo />` or `<NexFlowLogoCompact />`
- Background → `<LivingMesh />` once per authenticated layout

---

15. shadcn/ui Setup and Theme Tokens

Run after scaffold:

```
cd apps/web
pnpm dlx shadcn@latest init -d
```

```
pnpm dlx shadcn@latest add button input card dialog dropdown-menu tabs table badg
```

Accept defaults except:

- Style: `default`
- Base color: `slate`
- CSS variables: `yes`
- Import alias: `@/components`

---

16. Real-time — WebSockets + SSE

SSE for signal stream — `apps/api/src/routes/signals.ts` snippet

```
app.get("/stream", async (c) => {
  const orgId = c.get("orgId") as string;
  return new Response(
    new ReadableStream({
      async start(controller) {
        const enc = new TextEncoder();
        const send = (data: unknown) =>
          controller.enqueue(enc.encode(`data: ${JSON.stringify(data)}\n\n`));
        const interval = setInterval(async () => {
          const fresh = await db.query.signals.findMany({
            where: and(eq(signals.orgId, orgId), gte(signals.createdAt, new Date(
            orderBy: desc(signals.createdAt), limit: 10,
          });
          send(fresh);
        }, 5000);
        c.req.raw.signal.addEventListener("abort", () => clearInterval(interval))
      },
    }),
    { headers: { "Content-Type": "text/event-stream", "Cache-Control": "no-cache"
  );
});
```

---

17. Auth (Clerk or Auth.js)

Pack defaults to **Clerk** for speed. Webhook `POST /v1/webhooks/clerk` receives user

creation and mirrors into the `users` table, linking via `authId`.

apps/web/middleware.ts

```
import { clerkMiddleware, createRouteMatcher } from "@clerk/nextjs/server";

const isProtected = createRouteMatcher(["/((?!sign-in|sign-up|api/public).*)"]);
export default clerkMiddleware((auth, req) => {
  if (isProtected(req)) auth().protect();
});
export const config = { matcher: ["/((?!_next|.*\\..*).*)"] };
```

---

18. Job Queue (BullMQ + Redis)

apps/api/src/jobs/enqueue.ts

```
import { Queue } from "bullmq";
import Redis from "ioredis";

const connection = new Redis(process.env.REDIS_URL!, { maxRetriesPerRequest: null

export const enrichQueue = new Queue("enrich", { connection });
export const screenQueue = new Queue("screen", { connection });
export const scoreQueue = new Queue("score", { connection });

export const enqueueEnrich = (d: { contactId: string; orgId: string }) => enrichQ
export const enqueueScreen = (d: { contactId: string; orgId: string; trigger?: st
export const enqueueScore = (d: { contactId: string; orgId: string }) => scoreQue
export const enqueueWhatsappInbound = (body: unknown) =>
  new Queue("whatsapp_inbound", { connection }).add("wa_in", body);
```

apps/api/src/jobs/enrich.worker.ts

```
import { Worker } from "bullmq";
import Redis from "ioredis";
import { enrichContact } from "@nexflow/ai";

const connection = new Redis(process.env.REDIS_URL!, { maxRetriesPerRequest: null

new Worker("enrich", async (job) => {
  const { contactId, orgId } = job.data as { contactId: string; orgId: string };
```

```
  await enrichContact(contactId, orgId);
}, { connection, concurrency: 5 });
```

Mirror for `screen.worker.ts` and `score.worker.ts` using `screenContact` and `scoreContact` respectively.

Workers boot alongside API in `apps/api/src/index.ts`:

```
// near bottom, before serve()
import "./jobs/enrich.worker";
import "./jobs/screen.worker";
import "./jobs/score.worker";
```

---

19. Observability (Sentry + OpenTelemetry)

`apps/api/src/observability.ts`

```
import * as Sentry from "@sentry/node";
if (process.env.SENTRY_DSN) {
  Sentry.init({ dsn: process.env.SENTRY_DSN, tracesSampleRate: 0.1, environment:
}
export { Sentry };
```

Import this at the very top of `apps/api/src/index.ts` before any other imports.

---

20. Replit Runtime Files (complete)

`.replit` (root)

```
run = "pnpm dev"
entrypoint = "apps/api/src/index.ts"
hidden = [".turbo", "node_modules", ".next"]

[nix]
channel = "stable-24_05"

[env]
NODE_ENV = "development"
```

```
[packager]
language = "nodejs"

[[ports]]
localPort = 3000
externalPort = 80

[[ports]]
localPort = 4000
externalPort = 4000

[deployment]
run = ["sh", "-c", "pnpm build && pnpm start"]
deploymentTarget = "cloudrun"

[interpreter]
command = ["pnpm", "dev"]
```

replit.nix

```
{ pkgs }: {
  deps = [
    pkgs.nodejs_20
    pkgs.pnpm
    pkgs.postgresql_16
    pkgs.redis
    pkgs.openssl
  ];
}
```

---

21. Build & Deploy Commands

First run (Replit shell)

```
pnpm install
pnpm --filter @nexflow/db db:push                  # creates tables
psql $DATABASE_URL -f packages/db/src/policies.sql  # RLS + triggers
pnpm --filter @nexflow/db db:seed                   # demo org + admin
pnpm dev                                             # boots web + api + workers
```

Production build

```
pnpm build
pnpm start
```

Database operations

```
pnpm db:generate          # create new migration from schema diff
pnpm db:migrate           # apply pending migrations
pnpm db:studio            # open Drizzle Studio
```

---

22. Replit AI Master Prompt (paste verbatim)

> Paste the block between `>>>` and `<<<` into Replit AI as your first message.

```
>>>
You are scaffolding a production-grade monorepo for NexFlow — a universal AI-nati

RULES (non-negotiable):
1. Follow the build pack I am about to paste section by section. When I say "appl
2. Stack: Next.js 15 (App Router) + Hono API + PostgreSQL 16 + Drizzle ORM + Redi
3. Monorepo: pnpm workspaces + Turborepo. `apps/web`, `apps/api`, `packages/db`,
4. Every tenant-scoped table has `org_id` + RLS. Every money column is `bigint` i
5. `compliance_screenings` is append-only — install the Postgres trigger from Sec
6. AI calls go through the gateway in `packages/ai/gateway.ts`. Always filter Ant
7. JSON parsing of AI output strips ```json fences then `JSON.parse`.
8. Never commit secrets. All `.env` keys live in Replit Secrets.
9. The 17 frontend module pages follow the Contacts page pattern in Section 14. O
10. Do NOT add features I didn't ask for. Do NOT change the product from "univers

BUILD ORDER:
Step 1  — Initialize monorepo (Section 3 + Section 5).
Step 2  — Create `.env.example` (Section 4) and `.replit` + `replit.nix` (Section
Step 3  — Build `packages/shared` types (Section 10 types block).
Step 4  — Build `packages/db` schema (Section 6), run `db:push`, apply `policies.
Step 5  — Build `packages/ai` gateway + 10 agents (Section 9).
Step 6  — Build `apps/api`: middleware, all 15 routers (Section 8), WebSocket han
Step 7  — Build `apps/web`: Clerk auth (Section 17), tailwind + globals (Section
Step 8  — **Apply Section 14b brand identity end-to-end: import fonts, add Living
Step 9  — Seed demo org (Section 7).
Step 10 — Run `pnpm dev`, open `/sign-up`, create account, visit `/` (Home). The

After every step, report what you built and wait for "continue" before moving on.
```

```
I will paste Section 3 next.
<<<
```

After pasting that, paste **Sections 3 → 20 in order**, each prefixed with: `Apply Section N exactly.`

---

23. Post-Build Verification Checklist

Run through this list before you consider the build done.

- `GET http://localhost:4000/health` returns `{status:"ok"}`.
- `pnpm db:studio` opens and shows all 16 tables.
- Signing up via `/sign-up` creates a Clerk user AND a row in `users` (Clerk webhook wired).
- `POST /v1/contacts` with a valid body returns 201 and triggers enrichment + screening jobs (check BullMQ UI or logs).
- Visiting `/contacts` shows the contact. Clicking it opens `/contacts/[id]` (Profile 360°).
- `POST /v1/ai/chat` with `{ messages: [{role:"user", content:"hi"}] }` streams a response.
- Compliance: creating a contact auto-creates a `compliance_screenings` row. Attempting `DELETE FROM compliance_screenings` throws the append-only error.
- RLS: opening two orgs (A, B); A cannot see B's contacts via API.
- Call flow: `POST /v1/calls/start` with a valid Twilio number returns a `callId`; Twilio console shows the outbound call initiated.
- WhatsApp webhook: Meta's webhook verify handshake succeeds against `/webhooks/whatsapp/webhook`.
- Job queue: killing and restarting the API resumes pending jobs.
- Daily briefing: `GET /v1/ai/briefing/daily` returns a text brief under 200 words.
- Segment: `POST /v1/segments` with `{naturalLanguageQuery: "CTOs in fintech in UAE with leadScore > 60"}` returns a structured filter.
- Sentry: triggering an error in a route reports to Sentry dashboard.
- **Brand: Living Mesh nodes are visibly drifting in the background of every**

authenticated page.

- **Brand: the sidebar wordmark "NexFlow" cycles colors every 6 seconds (Chameleon).**
- **Brand: the `<NexFlowLogo />` component pulses at 2.4s and its three diamonds draw in sequentially on first load.**
- **Brand: primary CTAs render with the animated Chameleon gradient fill, not a flat color.**
- **Brand: the AI orb in the top-bar breathes (Pulse) and uses `Fraunces` italic "N".**
- **Brand: reduced-motion setting disables all looping animations while preserving the static palette.**
- **Brand: fonts load — DM Serif Display on headings, Inter/DM Sans on body, DM Mono on numbers.**

If every box is checked, you have a working NexFlow instance.

End of build pack. This document is self-contained. No external references, no placeholders. Every Section 6 column, every Section 8 route, every Section 9 agent has the full code required to build. If Replit AI drifts or asks questions, reply with: *"Follow the pack. Apply Section N verbatim."*